CO3020 - Computer Organization 計算機組織

Instructions: Language of the Computer

指令: 電腦的語言

江育綺

2026-09-30

NCU Communication Engineering



Outline - I

- Instruction Set
- Operands
  - Register Operands
  - Memory Operands
  - Immediate Operands
- Representing Data
  - Signed and Unsigned Numbers
- Representing Instructions
- Operations
  - Logical Operations
  - Instructions for Making Decisions

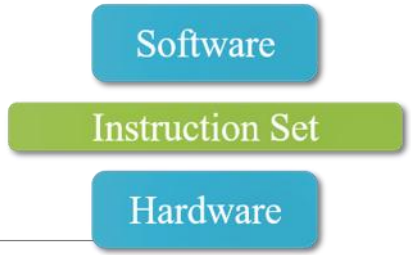
Outline - II

- Supporting Procedures in Computer Hardware
- Communicating with People
- MIPS Addressing for 32-Bit Immediates and Addresses
- Translating and Starting a Program
- A Sort Example
- Arrays versus Pointers
- ARM and x86 instruction sets
- Fallacies and Pitfalls
- Concluding Remarks

Instruction Set

CHAPTER2: INSTRUCTIONS: LANGUAGE OF THE COMPUTER

Instruction Set



- The collection of instructions that a CPU can understand and execute.
- Different computers have different instruction sets
  - But with many aspects in common
- Early computers had very simple instruction sets
  - Simplified implementation
- Many modern computers also have simple instruction sets
  - They used to develop CISC (complex)

➔Complicated hardware design and slower clock rate
➔Now, RISC (reduced)

The MIPS Instruction Set

- Used MIPS as the example throughout the book
  - Simple, representative, and easy to understand
- Stanford MIPS commercialized by MIPS Technologies (www.mips.com)
- Typical of many modern ISAs
  - See MIPS Reference Data tear-out card, and Appendixes B and E
- Similar ISAs have a large share of embedded core market
  - Applications in consumer electronics, network/storage equipment, cameras, printers, …

Different CPUs use different ISAs

- Different instruction sets have many aspects in common

CPU	ISA
Intel	x86
AMD	x86
Apple M-series	ARM
Raspberry Pi	ARM
Qualcomm Snapdragon	ARM
RISC-V boards	RISC-V

Concept	MIPS	ARM	x86	RISC-V
Addition	add	ADD	add	add
Subtraction	sub	SUB	sub	sub
Load	lw	LDR	mov*	lw
Store	sw	STR	mov*	sw
Branch if equal	beq	BEQ	je	beq
Jump / Call	j, jal	B, BL	jmp, call	jal

	x86	ARM	RISC-V	MIPS
Usage	High	High	Growing	Limited
App.	PC, Server	Mobile, Embedded, Apple Silicon	Embedded, IoT, Research	Legacy, Education



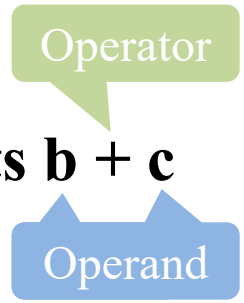
Operands

CHAPTER2: INSTRUCTIONS: LANGUAGE OF THE COMPUTER

Arithmetic Operations

- Add and subtract, three operands
  - **Two sources** and **one destination**
  - **add a, b, c** **# a gets b + c**

    (a = b + c)



- All arithmetic operations have this form

- CPU Design Principle 1: **Simplicity favors regularity**
  - Regularity makes implementation simpler
  - Simplicity enables higher performance at lower cost

Arithmetic Example

C code:

```
f = (g + h) - (i + j);
```

Compiled MIPS code:

```
add t0, g, h        # temp t0 = g + h
add t1, i, j        # temp t1 = i + j
sub f, t0, t1       # f = t0 - t1
```

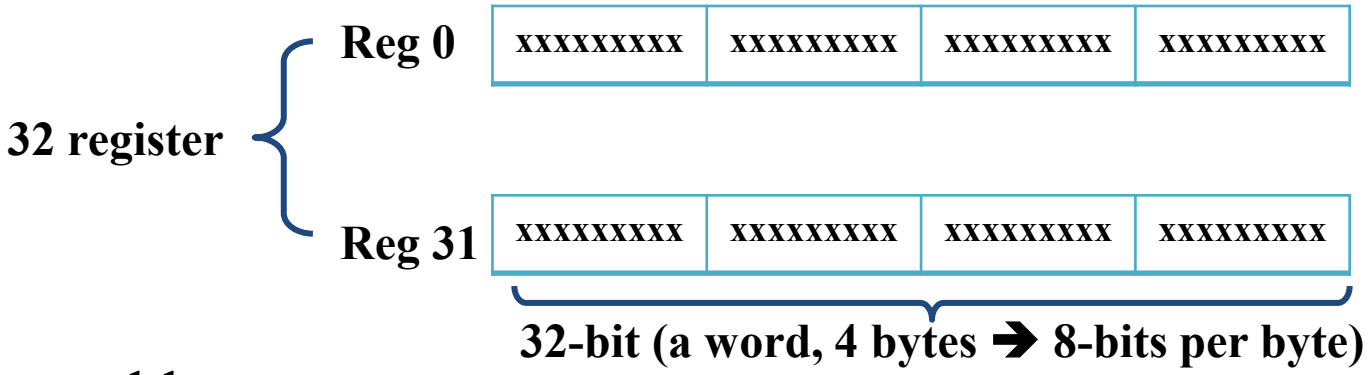
Register Operands

CHAPTER2: INSTRUCTIONS: LANGUAGE OF THE COMPUTER

Register Operands

Inside CPU | Very fast | Limited

- Arithmetic instructions are performed with **data in register**
- MIPS has a 32 × 32-bit register file ➔ 32-bit system CPU



- Assembler names
  - **$t0, $t1, …, $t9 for temporary values ➔ $8 – $15**
  - **$s0, $s1, …, $s7 for saved variables ➔ $16 – 23**

- CPU Design Principle 2: **Smaller is faster**
  - Memory is much larger than register

What's the size for CPU register?

Numbered Register

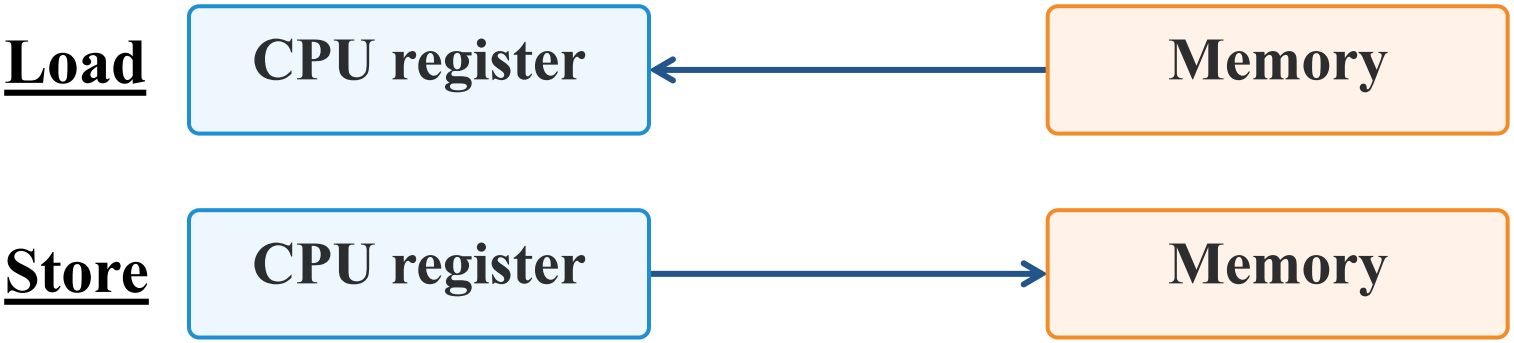
Number	Register	Usage
0	$zero	Constant 0
1	$at	Reserved for assembler
2–3	$v0–$v1	Return values
4–7	$a0–$a3	Function arguments
8–15	$t0–$t7	Temporary (caller-saved)
16–23	$s0–$s7	Saved (callee-saved)
24–25	$t8–$t9	Temporary (caller-saved)
26–27	$k0–$k1	Reserved for OS kernel
28	$gp	Global pointer
29	$sp	Stack pointer
30	$fp	Frame pointer
31	$ra	Return address

Memory Operands

CHAPTER2: INSTRUCTIONS: LANGUAGE OF THE COMPUTER

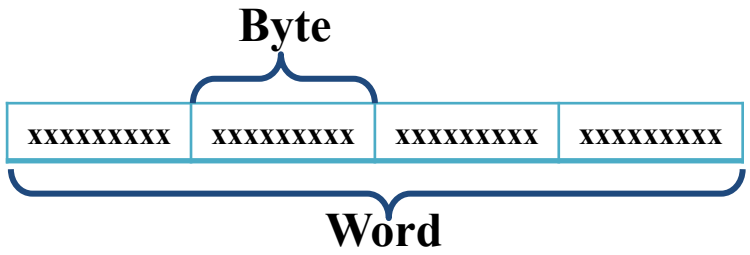
Memory Operands (1/4)

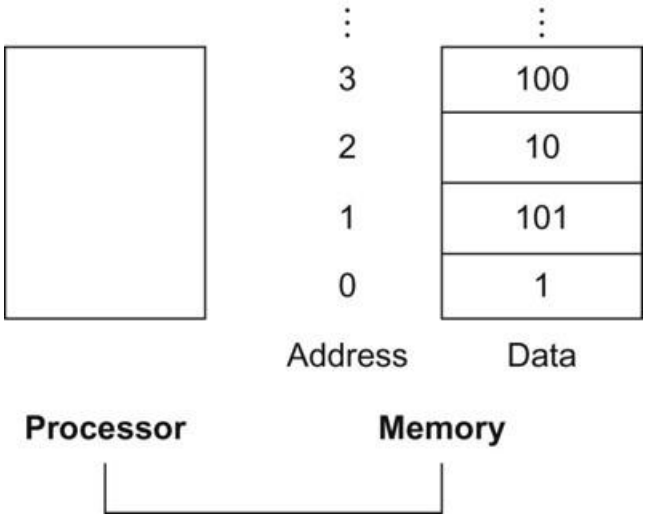
- Main memory used for composite data
  - Arrays, structures, dynamic data, e.g., **array A[10]**
- To apply arithmetic operations (performed on registers)
  - **Load** values **from memory into registers**
  - **Store** result **from register to memory**



Memory Operands (2/6)

- Memory is **byte addressed**
  - Each address identifies an **8-bit byte**
  - Access data requires **base register (array)** and **offset (index)**





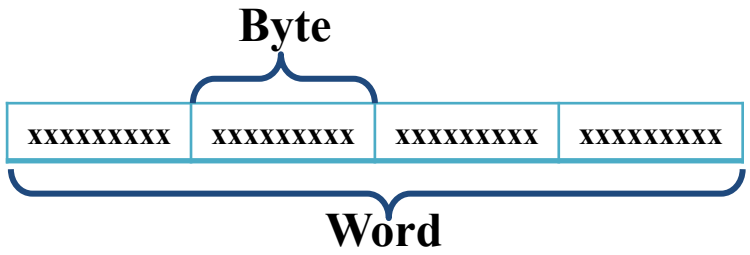

```
Memory[2] = 10

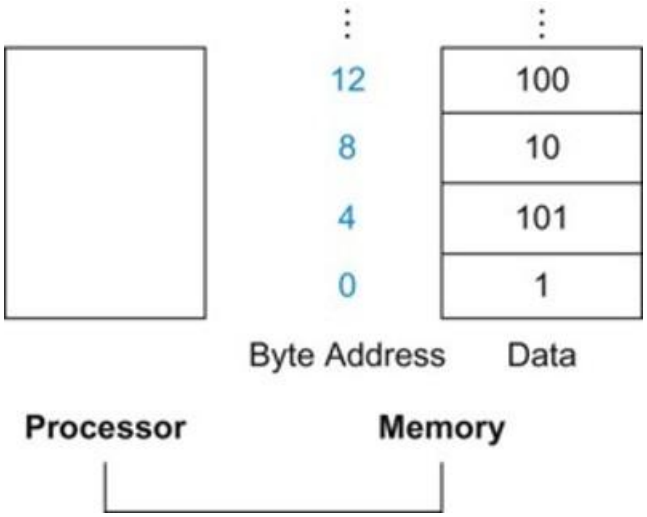
Memory array A      ➔Base register
A[0] = Memory[0]    ➔Base register+0

Memory[2]           ➔Offset = 2
                    ➔Base + offset
```

Memory Operands (3/6)

- Memory is **byte addressed**
  - Each address identifies an **8-bit byte**
  - Access data requires **base register (array)** and **offset (index)**






```
Memory[0+4*2] = 10

Memory array A      ➔Base register
A[0] = Memory[0]    ➔Base register+0

Memory[2]           ➔Offset = 4*2= 8
                    ➔Base + offset
```

Memory Operands (4/6)

- MIPS uses a 32-bit address.

ONE Address Value

00010000	00000001	00000000	00000000

32 bits represent ONE address number

Smallest

00000000	00000000	00000000	00000000

Address value = 0

Largest

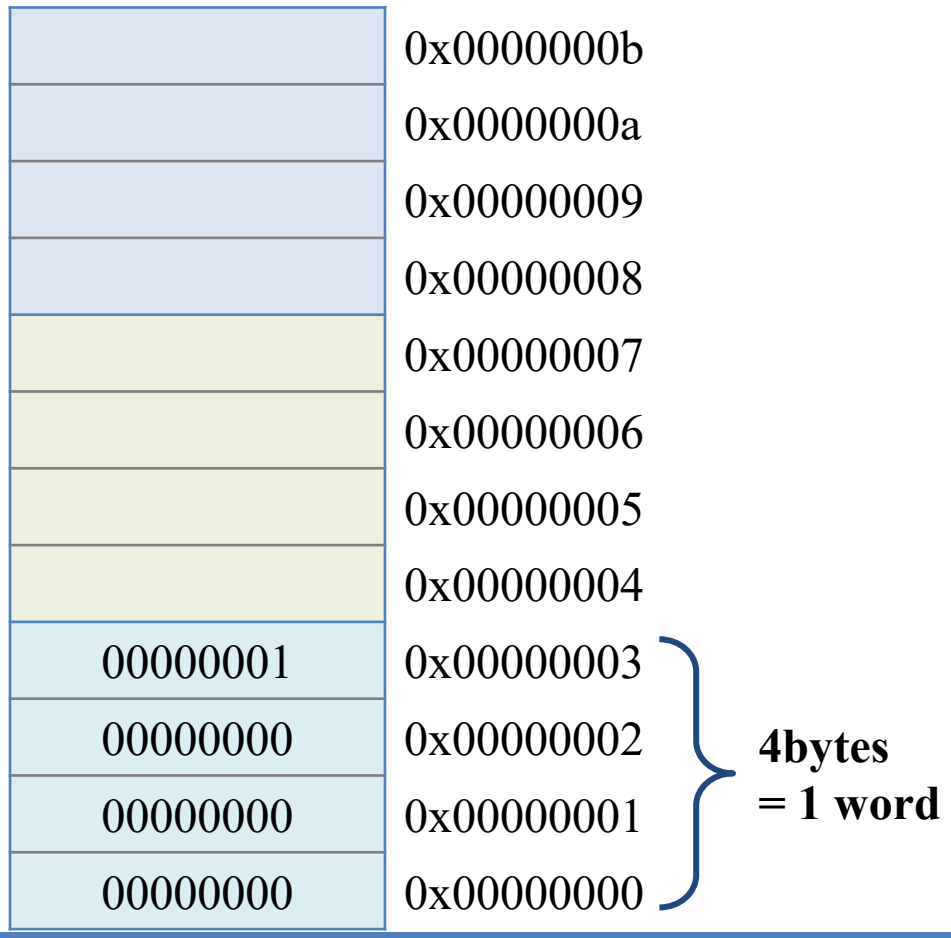
11111111	11111111	11111111	11111111

Address value = $2^{32} - 1$

32-bit address → 2^{32} possible address values

Memory Operands (5/6)

- MIPS uses these 2^{32} address values to identify byte-addressable memory locations.



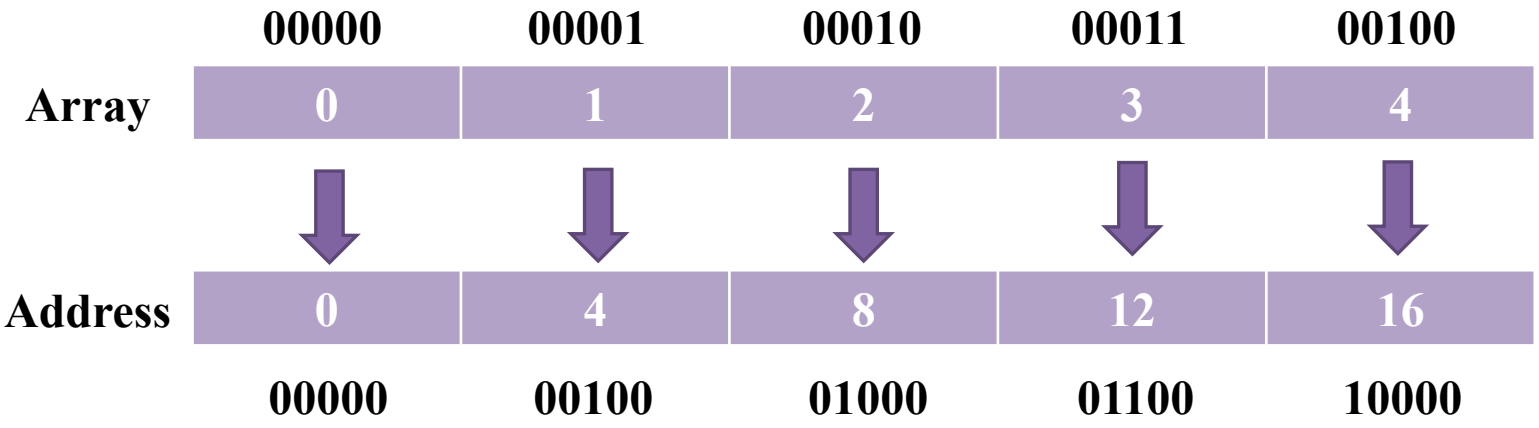
How many words?
$\frac{2^{32}}{4} = 2^{30}$ words

==Address Space Size==
2^{32} addresses
$\times$ 1 byte/address
$= 2^{32}$ bytes $= 4$GiB

How are the 4 bytes ordered?
➔Big-endian / Little-endian

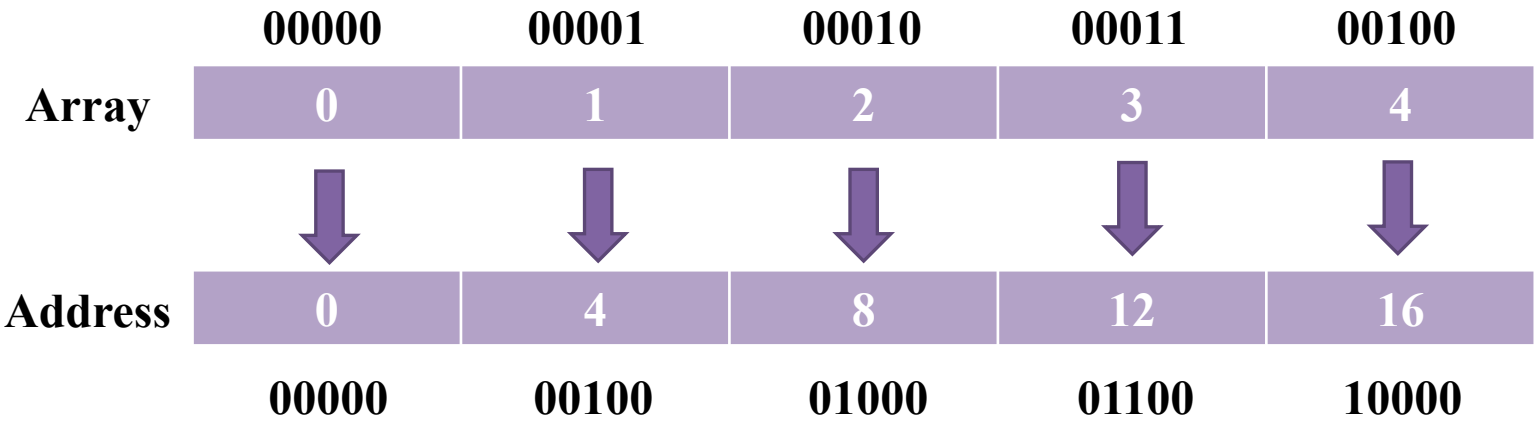
Memory Operands (6/6)

- Array Index vs. Byte Address
  - Array index is not a memory address. The CPU must convert the index into a byte address by multiplying it by the size of each element.



Memory Operands (6/6)

- Array Index vs. Byte Address
  - Array index is not a memory address. The CPU must convert the index into a byte address by multiplying it by the size of each element.



- **Alignment restriction:** In MIPS, words must start at addresses that are multiples of 4.

Memory Operand Example 1

C code:

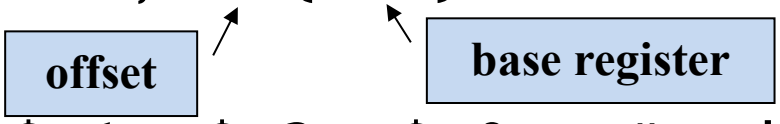
```
g = h + A[8];
```

- g in $s1, h in $s2, base address of A in $s3

Compiled MIPS code:

- Index 8 requires offset of 32
  - 4 bytes per word

```
lw  $t0, 32($s3)    # load word, tmp = A[8]
add $s1, $s2, $t0   # addition, g = h + tmp
```



Copy words between memory and register in MIPS

Memory Operand Example 2

C code:

```
A[12] = h + A[8];
```

- h in $s2, base address of A in $s3

Compiled MIPS code:

- Index 8 requires offset of 32

```
lw  $t0, 32($s3)    # load word
add $t0, $s2, $t0
sw  $t0, 48($s3)    # store word
```

Registers vs. Memory

- Registers are faster to access than memory
  - CPU Design Principle 2: Smaller is faster
- Operating on memory data requires loads and stores
  - More instructions to be executed

Registers vs. Memory

- Registers are faster to access than memory
  - CPU Design Principle 2: Smaller is faster
- Operating on memory data requires loads and stores
  - More instructions to be executed
- Compiler must use registers for variables as much as possible
  - ***Spilling registers:*** The process of putting less frequently used variables into memory (or those needed later)
  - Register optimization is important! (small size)

Registers vs. Memory

- Registers are faster to access than memory
  - CPU Design Principle 2: Smaller is faster
- Operating on memory data requires loads and stores
  - More instructions to be executed
- Compiler must use registers for variables as much as possible
  - ***Spilling registers:*** The process of putting less frequently used variables into memory (or those needed later)
  - Register optimization is important! (small size)
- Data is more useful when in register (higher throughput)
  - In MIPS, arithmetic instructions can read two registers, operate on them, and write the result.
  - In MIPS, data transfer instructions only reads one operand or writes one operand, without operating on it.

Immediate Operands

CHAPTER2: INSTRUCTIONS: LANGUAGE OF THE COMPUTER

Immediate Operands

- In SPEC CPU2006 benchmarks, **more than a half** instructions have a constant as an operand.
- Specified typical constants in memory to load

```
addi $s3, $s3, 4
```

- No subtract immediate instruction
  - Just use a negative constant

```
addi $s2, $s1, -1
```

- Design Principle 3: **Make the common case fast**
  - Small constants are common
  - Immediate operand avoids a load instruction

The Constant Zero

- The number zero is frequently used in code. It's defined in general-usage register
- MIPS register 0 ($zero) is the constant 0
  - Cannot be overwritten
- Useful for common operations
  - E.g., move between registers

```
add $t2, $s1, $zero  # move $s1 to $t2, $t2 = $s1
```

  - E.g., bitwise NOT operation

```
nor $t0, $t1, $zero  # $t0 = ~$t1
```

Appendix: MIPS Assembly Language

- Arithmetic

Description	Instruction	Meaning
Add	`add $s1,$s2,$s3`	`$s3 = $s1 + $s2`
Subtract	`sub $s1,$s2,$s3`	`$s3 = $s1 - $s2`
Add Immediate	`addi $s1,$s2,20`	`$s2 = $s1 + 20`

Appendix: MIPS Assembly Language

- Memory Access (Data Transfer)

Description	Instruction	Meaning
Load Word	`lw $s1,20($s2)`	`$s1 = Memory[$s2 + 20]`
Store Word	`sw $s1,20($s2)`	`Memory[$s2 + 20] = $s1`
Load Halfword	`lh $s1,20($s2)`	`$s1 = Memory[$s2 + 20]` *(16-bit)*
Load Halfword Unsigned	`lhu $s1,20($s2)`	`$s1 = Memory[$s2 + 20]` *(16-bit unsigned)*
Store Halfword	`sh $s1,20($s2)`	`Memory[$s2 + 20] = $s1` *(16-bit)*
Load Byte	`lb $s1,20($s2)`	`$s1 = Memory[$s2 + 20]` *(8-bit)*
Load Byte Unsigned	`lbu $s1,20($s2)`	`$s1 = Memory[$s2 + 20]` *(8-bit unsigned)*
Store Byte	`sb $s1,20($s2)`	`Memory[$s2 + 20] = $s1` *(8-bit)*
Load Upper Immediate	`lui $s1,20`	`$s1 = 20 x 2^{16}`

Appendix: MIPS Assembly Language

- Logical

Description	Instruction	Meaning
Bitwise AND	`and $s1,$s2,$s3`	`$s1 = $s2 & $s3`
Bitwise OR	`or $s1,$s2,$s3`	`$s1 = $s2 | $s3`
Bitwise NOR	`nor $s1,$s2,$s3`	`$s1 = ~($s2 | $s3)`
AND Immediate	`andi $s1,$s2,20`	`$s1 = $s2 & 20`
OR Immediate	`ori $s1,$s2,20`	`$s1 = $s2 | 20`
Shift Left Logical	`sll $s1,$s2,10`	`$s1 = $s2 << 10`
Shift Right Logical	`srl $s1,$s2,10`	`$s1 = $s2 >> 10`

Appendix: MIPS Assembly Language

- Branch

Description	Instruction	Meaning
Branch if Equal	`beq $s1,$s2,label`	`if ($s1 == $s2) goto label`
Branch if Not Equal	`bne $s1,$s2,label`	`if ($s1 ≠ $s2) goto label`
Set on Less Than	`slt $s1,$s2,$s3`	`if ($s2 < $s3) $s1 = 1; else $s1 = 0`
Set on Less Than (Unsigned)	`sltu $s1,$s2,$s3`	`Unsigned comparison`
Set on Less Than Immediate	`slti $s1,$s2,20`	`if ($s2 < 20) $s1 = 1; else 0`
Set on Less Than Immediate (Unsigned)	`sltiu $s1,$s2,20`	`Unsigned immediate comparison`

Representing Numbers

CHAPTER2: INSTRUCTIONS: LANGUAGE OF THE COMPUTER

Unsigned Binary Integers

In any number base, the value of ith digit d is:

$$d \times \text{Base}^{i}$$

Given an n-bit number

$$x = x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \cdots + x_1 2^1 + x_0 2^0$$

- Range: 0 to $+2^n - 1$
- Example
  - $0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1011_2$
    $= 0 + \ldots + 1\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0$
    $= 0 + \ldots + 8 + 0 + 2 + 1 = 11_{10}$
- Using 32 bits
  - 0 to +4,294,967,295

2s-Complement Signed Integers

- 2's complement: 1's complement + 1
- $(-4)_{10}$

$1 - 0 = 1$
$1 - 1 = 0$

Inverted: $1 - x = \bar{x}$

➔ $\overline{(0100)_2}$ ➔ $(1011)_2 + 1 = (1100)_2$

	Positive Integers		Negative Numbers	
			Sign and	2's Complement
$+N$	(all systems)	$-N$	Magnitude	N^*
+0	0000	−0	1000	——
+1	0001	−1	1001	1111
+2	0010	−2	1010	1110
+3	0011	−3	1011	1101
+4	0100	−4	1100	1100
+5	0101	−5	1101	1011
+6	0110	−6	1110	1010
+7	0111	−7	1111	1001
		−8	——	1000

magnitude range is
$-(2^{n-1}) \sim (2^{n-1} - 1)$
$n = 4$, range is $-8 \sim 7$

2s-Complement Signed Integers

Given an n-bit number

$$x = -x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \cdots + x_1 2^1 + x_0 2^0$$

- Range: -2^{n-1} to $+2^{n-1} - 1$
- Using 32 bits
  - –2,147,483,648 to +2,147,483,647
- Bit 31 is sign bit
  - 1 for negative numbers
  - 0 for non-negative numbers
- Non-negative numbers have the same unsigned and 2s-complement representation

$$\overline{x} + 1 = -x$$
$$x + \overline{x} = 1111\ldots111_2 = -1$$

2^{n-1} can't be represented

2s-Complement Signed Integers

- Some specific numbers
  - 0: 0000 0000 … 0000
  - –1: 1111 1111 … 1111
  - Most-negative: 1000 0000 … 0000
  - Most-positive: 0111 1111 … 1111

$\overline{x} + 1 = -x$
➔$\overline{x} = -(x + 1)$
➔$\overline{-x} = -(-x + 1) = x - 1$
➔$\overline{-x} + 1 = x$

- Example
  - $1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1100_2$
    $= -1 \times 2^{31} + 1 \times 2^{30} + \ldots + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$
    $= -2{,}147{,}483{,}648 + 2{,}147{,}483{,}644 = -4_{10}$

Sign Extension

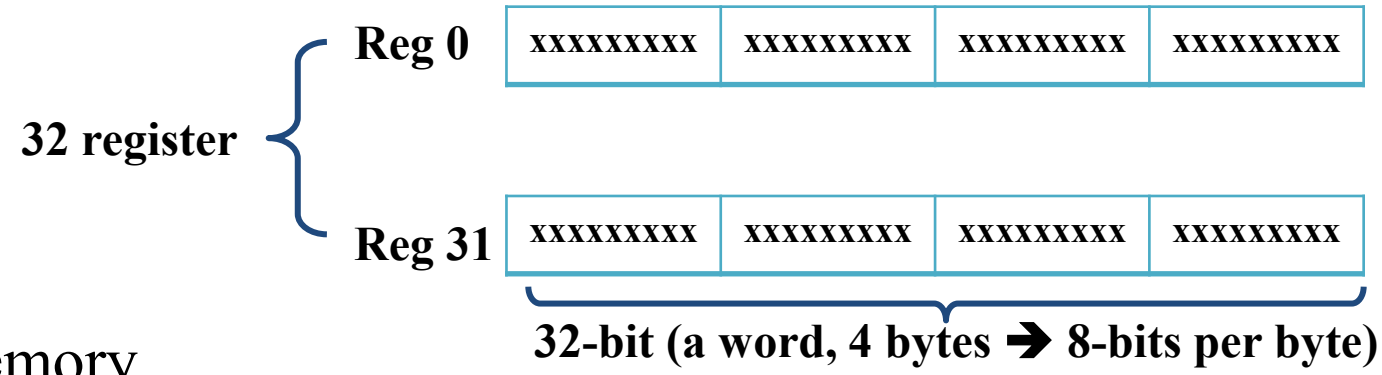
- Representing a number using more bits
  - Preserve the numeric value
- In MIPS instruction set
  - `addi`: extend immediate value
  - `lb, lh`: extend loaded byte/halfword
  - `beq, bne`: extend the displacement
- Replicate the sign bit to the left
  - c.f. unsigned values: extend with 0s
- Examples: 8-bit to 16-bit
  - +2: **0**000 0010 => **0000 0000 0**000 0010
  - –2: **1**111 1110 => **1111 1111 1**111 1110

Review of last week…

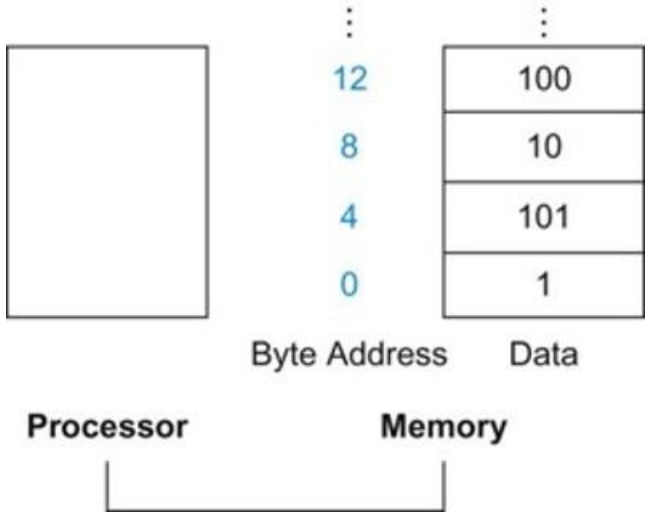
Operands

- # a gets b + c

- Register



- Memory
  - Byte-address



- Immediate

Memory Operands (4/6)

- MIPS uses a 32-bit address.

ONE Address Value

00010000	00000001	00000000	00000000

32 bits represent ONE address number

Smallest

00000000	00000000	00000000	00000000

Address value = 0

Largest

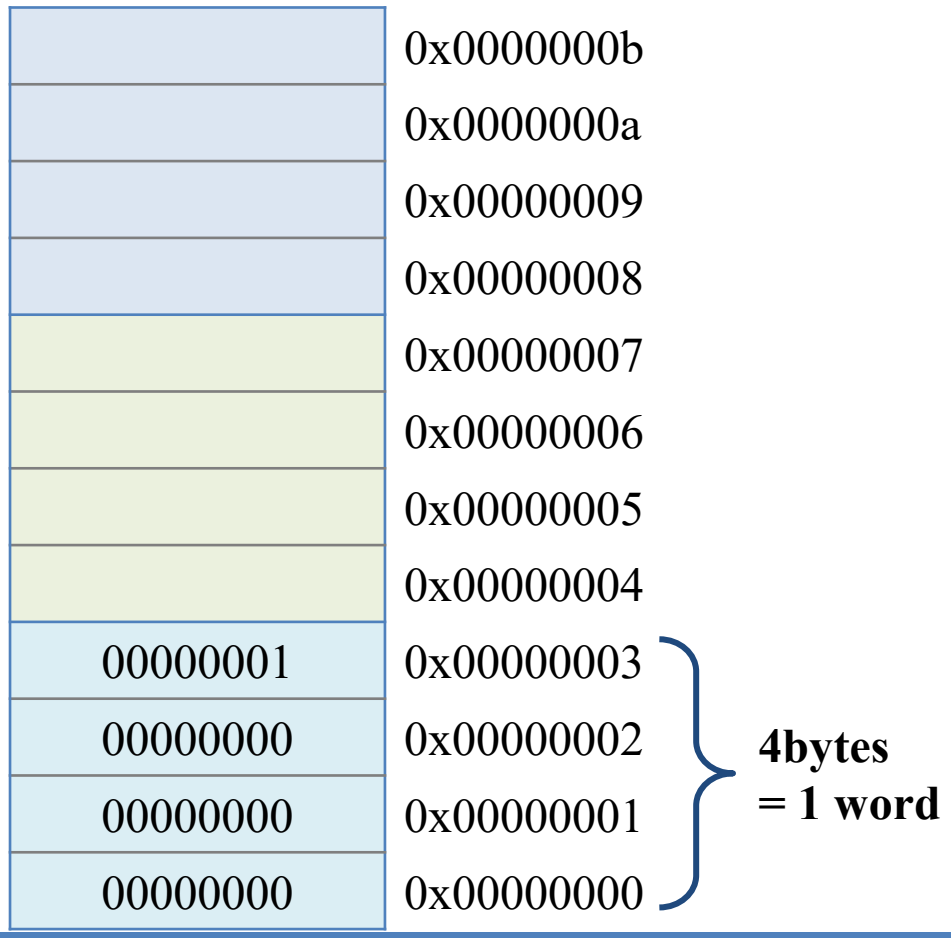
11111111	11111111	11111111	11111111

Address value = $2^{32} - 1$

32-bit address → 2^{32} possible address values

Memory Operands (5/6)

- MIPS uses these 2^{32} address values to identify byte-addressable memory locations.



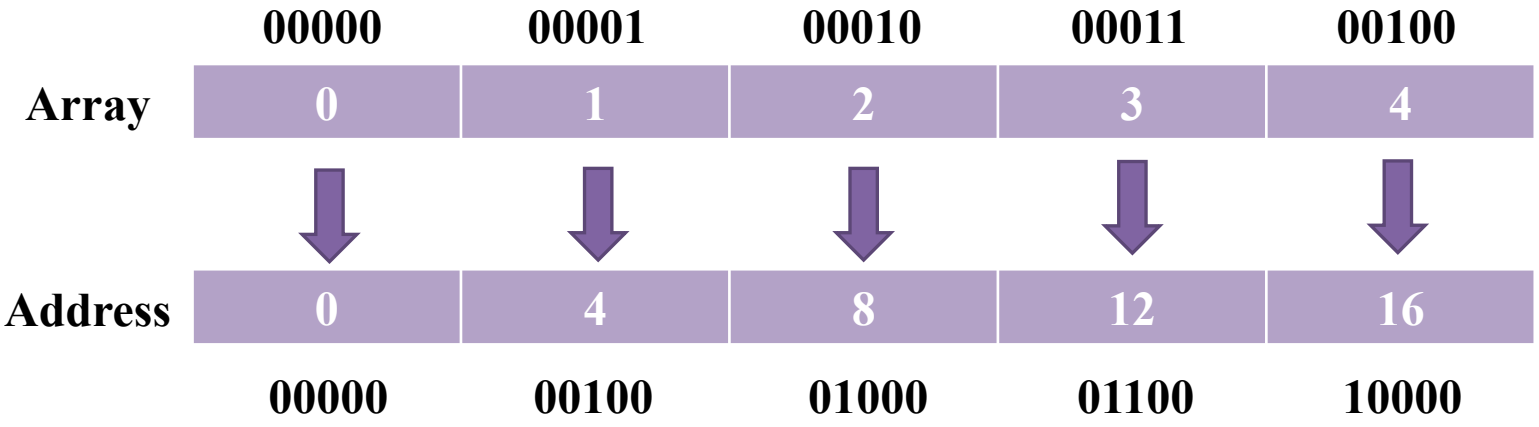
How many words?
$\frac{2^{32}}{4} = 2^{30}$ words

==Address Space Size==
2^{32} addresses
$\times$ 1 byte/address
$= 2^{32}$ bytes $= 4$GiB

How are the 4 bytes ordered?
➔Big-endian / Little-endian

Memory Operands (6/6)

- Array Index vs. Byte Address
  - Array index is not a memory address. The CPU must convert the index into a byte address by multiplying it by the size of each element.



- **Alignment restriction:** In MIPS, words must start at addresses that are multiples of 4.

Immediate Operands

- In SPEC CPU2006 benchmarks, **more than a half** instructions have a constant as an operand.
- Specified typical constants in memory to load

```
addi $s3, $s3, 4
```

- No subtract immediate instruction
  - Just use a negative constant

```
addi $s2, $s1, -1
```

- Design Principle 3: **Make the common case fast**
  - Small constants are common
  - Immediate operand avoids a load instruction

2s-Complement Signed Integers

Given an n-bit number

$$x = -x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \cdots + x_1 2^1 + x_0 2^0$$

- Range: -2^{n-1} to $+2^{n-1} - 1$
- Using 32 bits
  - –2,147,483,648 to +2,147,483,647
- Bit 31 is sign bit
  - 1 for negative numbers
  - 0 for non-negative numbers
- Non-negative numbers have the same unsigned and 2s-complement representation

$$\overline{x} + 1 = -x$$
$$x + \overline{x} = 1111\ldots111_2 = -1$$

2^{n-1} can't be represented

Sign Extension

- Representing a number using more bits
  - Preserve the numeric value
- In MIPS instruction set
  - `addi`: extend immediate value
  - `lb, lh`: extend loaded byte/halfword
  - `beq, bne`: extend the displacement
- Replicate the sign bit to the left
  - c.f. unsigned values: extend with 0s
- Examples: 8-bit to 16-bit
  - +2: **0**000 0010 => **0000 0000 0**000 0010
  - –2: **1**111 1110 => **1111 1111 1**111 1110

Representing Instructions

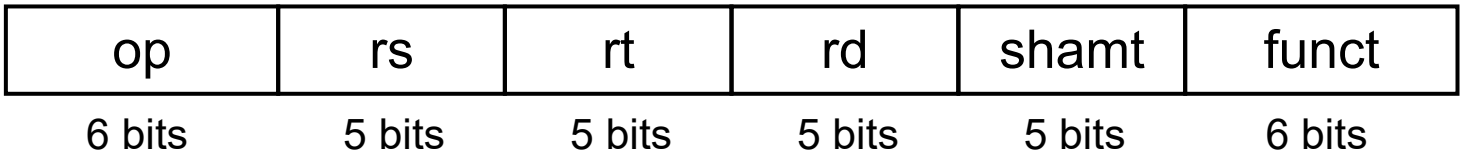
CHAPTER2: INSTRUCTIONS: LANGUAGE OF THE COMPUTER

Representing Instructions

- Instructions are encoded in binary
  - Called machine code
- MIPS instructions
  - Encoded as **32-bit instruction words in 3 different formats (R-, I-, and J-formats)**
- Register numbers
  - $t0 – $t7 are reg's 8 – 15
  - $s0 – $s7 are reg's 16 – 23
  - $t8 – $t9 are reg's 24 – 25

Number	Register	Usage
0	$zero	Constant 0
1	$at	Reserved for assembler
2–3	$v0–$v1	Return values
4–7	$a0–$a3	Function arguments
8–15	$t0–$t7	Temporary (caller-saved)
16–23	$s0–$s7	Saved (callee-saved)
24–25	$t8–$t9	Temporary (caller-saved)
26–27	$k0–$k1	Reserved for OS kernel
28	$gp	Global pointer
29	$sp	Stack pointer
30	$fp	Frame pointer
31	$ra	Return address

MIPS R-format Instructions



- R-type (for register) Instruction fields
  - op: operation code (opcode)
  - rs: first source register number
  - rt: second source register number
  - rd: destination register number
  - shamt: shift amount (00000 for now)
  - funct: function code (extends opcode)

```
e.g.,
add $rd, $rs, $rt
sub $rd, $rs, $rt
```

MIPS instruction coding

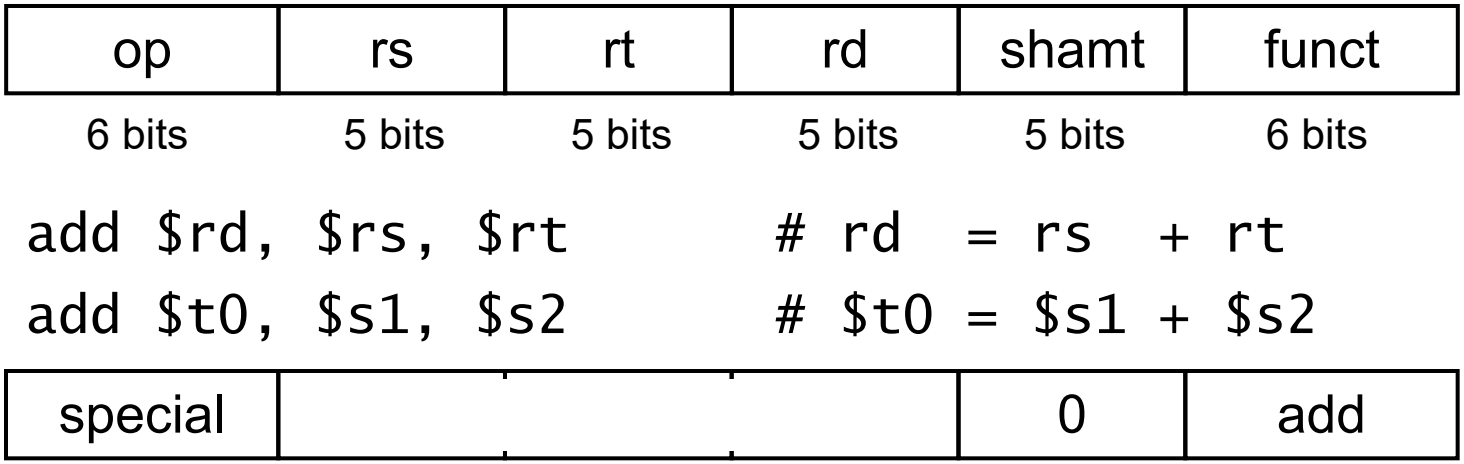
op(31:26)								
28–26 31–29	0(000)	1(001)	2(010)	3(011)	4(100)	5(101)	6(110)	7(111)
0(000)	R-format	Bltz/gez	jump	jump & link	branch eq	branch ne	blez	bgtz
1(001)	add immediate	addiu	set less than imm.	set less than imm. unsigned	andi	ori	xori	load upper immediate
2(010)	TLB	FlPt						
3(011)								
4(100)	load byte	load half	lwl	load word	load byte unsigned	load half unsigned	lwr	
5(101)	store byte	store half	swl	store word			swr	
6(110)	load linked word	lwc1						
7(111)	store cond. word	swc1						

MIPS instruction coding

op(31:26)=010000 (TLB), rs(25:21)								
23–21 25–24	0(000)	1(001)	2(010)	3(011)	4(100)	5(101)	6(110)	7(111)
0(00)	mfc0		cfc0		mtc0		ctc0	
1(01)								
2(10)								
3(11)								

op(31:26)=000000 (R-format), funct(5:0)								
2–0 5–3	0(000)	1(001)	2(010)	3(011)	4(100)	5(101)	6(110)	7(111)
0(000)	shift left logical		shift right logical	sra	sllv		srlv	srav
1(001)	jump register	jalr			syscall	break		
2(010)	mfhi	mthi	mflo	mtlo				
3(011)	mult	multu	div	divu				
4(100)	add	addu	subtract	subu	and	or	xor	not or (nor)
5(101)			set l.t.	set l.t. unsigned				
6(110)								
7(111)								

R-format Example



Number	Register	Usage
0	$zero	Constant 0
1	$at	Reserved for assembler
2–3	$v0–$v1	Return values
4–7	$a0–$a3	Function arguments
8–15	$t0–$t7	Temporary (caller-saved)
16–23	$s0–$s7	Saved (callee-saved)
24–25	$t8–$t9	Temporary (caller-saved)
26–27	$k0–$k1	Reserved for OS kernel
28	$gp	Global pointer
29	$sp	Stack pointer
30	$fp	Frame pointer
31	$ra	Return address

R-format Example

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

```
add $rd, $rs, $rt        # rd  = rs  + rt
add $t0, $s1, $s2        # $t0 = $s1 + $s2
```

special				0	add
0	17	18	8	0	32
000000	10001	10010	01000	00000	100000

$0000\ 0010\ 0011\ 0010\ 0100\ 0000\ 0010\ 0000_2 = 02324020_{16}$

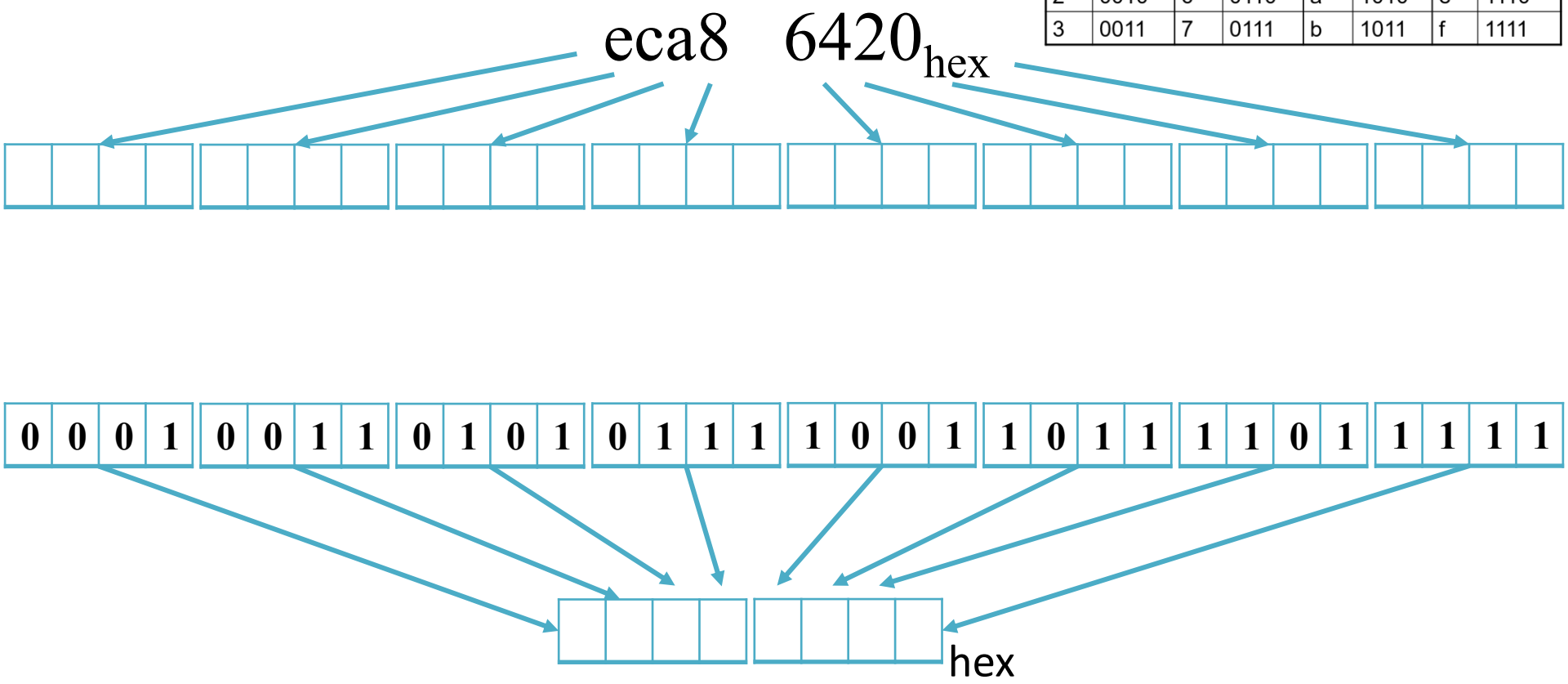
Hexadecimal

- Base 16
  - Compact representation of bit strings
  - 4 bits per hex digit

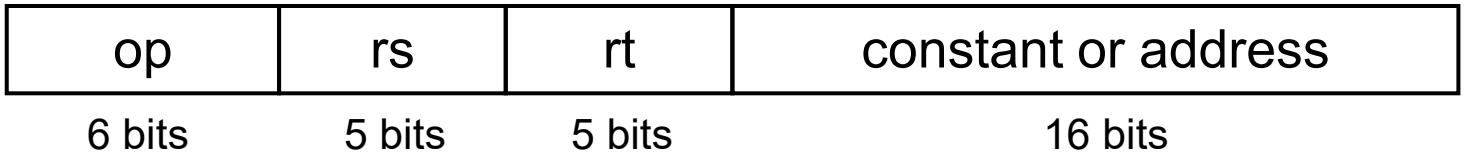
0	0000	4	0100	8	1000	c	1100
1	0001	5	0101	9	1001	d	1101
2	0010	6	0110	a	1010	e	1110
3	0011	7	0111	b	1011	f	1111

Hexadecimal - Example

0	0000	4	0100	8	1000	c	1100
1	0001	5	0101	9	1001	d	1101
2	0010	6	0110	a	1010	e	1110
3	0011	7	0111	b	1011	f	1111



MIPS I-format Instructions



- **Immediate arithmetic** and **data transfer load/store** instructions
  - **rt (register target)**: **destination or source** register number
  - Constant–2^{15} to $+2^{15} - 1$
  - **Address**: offset added to base address in **rs** (register source)
- Design Principle 4: **Good design demands good compromises**
  - Different formats complicate decoding, but allow 32-bit instructions uniformly
  - Keep formats as similar as possible

I-format Example

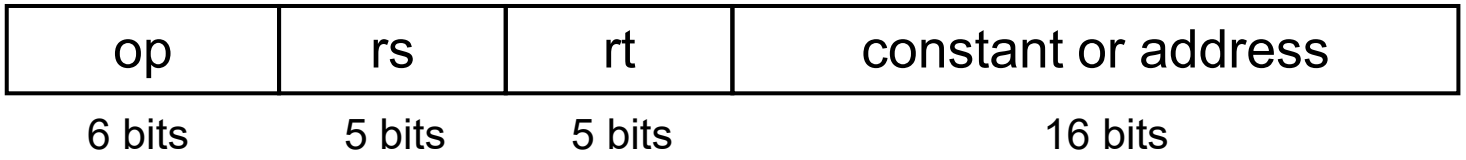
op	rs	rt	constant or address
6 bits	5 bits	5 bits	16 bits

```
addi rt, rs, immediate   # rt  = rs  + immediate
addi $s1, $s2, 100       # $s1 = $s2 + 100
```

Signed extended

addi	$s2	$s1	100
8_{hex}=8	18	17	100
001000	10010	10001	0000 0000 0110 0100

I-format Example

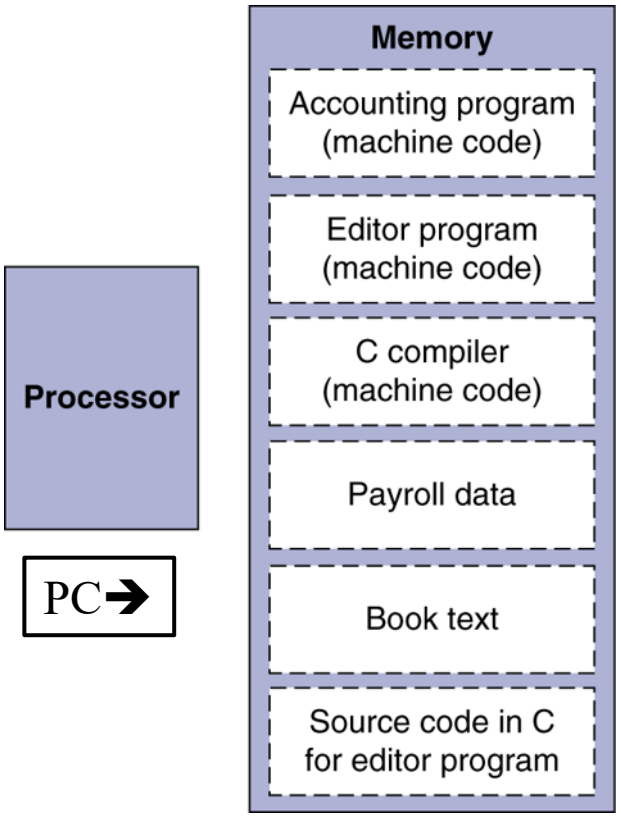



```
lw rt, offset(rs) # rt  = Mem[rs + offset]
lw $t0, $32($s3)  # Temporary reg $t0 gets A[8]
```

lw	$s3	$t0	32
23_{hex}=35	18	8	32
010011	10010	01000	0000 0000 0010 0000

Stored-Program Computers

The BIG Picture



- **Computers are built on 2 key principles**
  - Instructions represented as numbers, just like data
  - Instructions and data stored in memory, just like numbers
- **Everything can be stored in memory**
  - Everything has a memory address: instructions, data
- One register **keeps address of the instruction** being executed: **Program Counter (PC)**
  - Instruction address pointer
- **Binary compatibility** allows compiled programs to **work on different computers**
  - **Standardized ISAs**

Logical Operations

CHAPTER2: INSTRUCTIONS: LANGUAGE OF THE COMPUTER

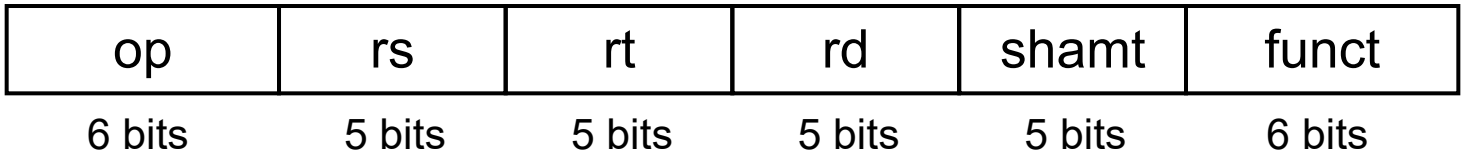
Logical Operations

Instructions for bitwise manipulation

Operation	C	Java	MIPS
Shift left	<<	<<	sll
Shift right	>>	>>>	srl
Bitwise AND	&	&	and, andi
Bitwise OR	|	|	or, ori
Bitwise NOT	~	~	nor

- Useful for extracting and inserting groups of bits in a word

Shift Operations



- **shamt: how many positions to shift**
- **Shift left logical (sll)**
  - Shift left and fill with 0 bits
  - `sll` by *i* bits multiplies by 2^i

```
sll $t2, $s0, 4      # reg $t2 = reg $s0 << 4 bits
```

0000 0000 0000 0000 0000 0000 0000 1001 = 9
0000 0000 0000 0000 0000 0000 1001 0000 = 144

- **Shift right logical (srl)**
  - Shift right and fill with 0 bits
  - `srl` by *i* bits divides by 2^i (unsigned only)

Range?

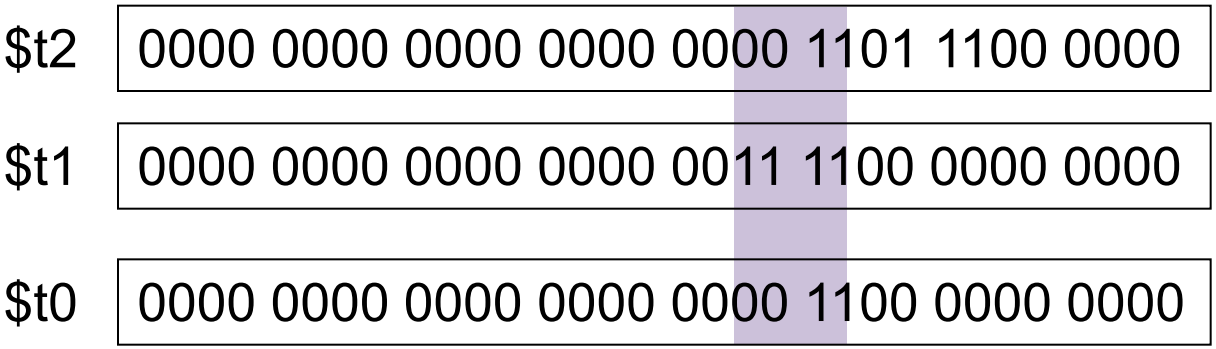
Might lost information of high position

Cannot retain the fraction

AND Operations

- **Useful to mask bits in a word**
  - Select some bits, clear others to 0

```
and $t0, $t1, $t2
```

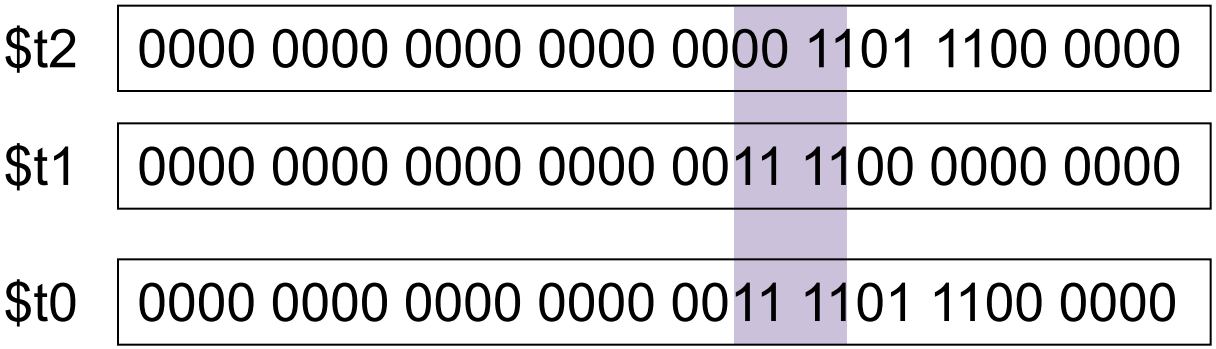


X AND 0 = 0
X AND 1 = X

OR Operations

- **Useful to include bits in a word**
  - Set some bits to 1, leave others unchanged

```
or $t0, $t1, $t2
```



X OR 0 = X
X OR 1 = 1

Cannot override the data 1 to 0

NOT Operations

- **Useful to invert bits in a word**
  - Change 0 to 1, and 1 to 0
- **MIPS has NOR 3-operand instruction**
  - a NOR b == NOT (a OR b)

```
nor $t0, $t1, $zero
```

Register 0: always read as zero

$t1	0000 0000 0000 0000 0011 1100 0000 0000
$t0	1111 1111 1111 1111 1100 0011 1111 1111

Instructions for Making Decisions

CHAPTER2: INSTRUCTIONS: LANGUAGE OF THE COMPUTER

Conditional Operations

- **Branch to a labeled instruction if a condition is true**
  - Otherwise, continue sequentially
- `beq rs, rt, L1 #branch if equal`
  - if (rs == rt) branch to instruction labeled L1;
- `bne rs, rt, L1 #branch if not equal`
  - if (rs != rt) branch to instruction labeled L1;
- `j L1 #=beq $0, $0, L1`
  - **Unconditional jump** to instruction labeled L1

I Type

J Type

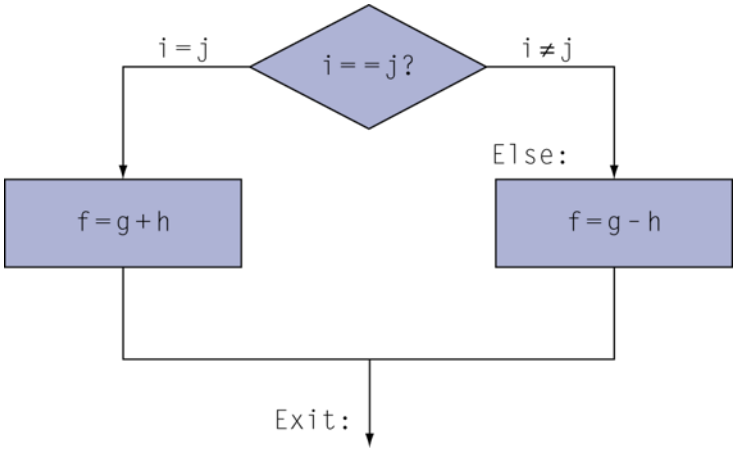


Compiling If Statements

C code:

```
if (i==j) f = g+h;
else f = g-h;
```

- f, g, h, i, j in $s0, $s1, $s2, $s3, $s4,



Compiled MIPS code:

```
        bne $s3, $s4, Else
        add $s0, $s1, $s2
        j   Exit
Else:   sub $s0, $s1, $s2
Exit:   …
```

Assembler calculates addresses

Compiling Loop Statements

C code:

```
while (save[i] == k) i += 1;
```

- i in $s3, k in $s5, address of save in $s6

Compiled MIPS code:

```
Loop: sll  $t1, $s3, 2
      add  $t1, $t1, $s6
      lw   $t0, 0($t1)
      bne  $t0, $s5, Exit
      addi $s3, $s3, 1
      j    Loop
Exit: …
```

Offset

Base + Offset

More Conditional Operations

- **Set result to 1 if a condition is true**
  - Otherwise, set to 0
  - set on less than (slt), set on less than immediate (slti)
  - set on less than unsigned (sltu), set on less than immediate unsigned (sltui)

`slt rd, rs, rt`
  - if (rs < rt) rd = 1; else rd = 0;

`slti rt, rs, constant`
  - if (rs < constant) rt = 1; else rt = 0;

- **Use in combination with `beq, bne`**

```
slt $t0, $s1, $s2  #     if ($s1 < $s2)
bne $t0, $zero, L  #          branch to L
```

Signed vs. Unsigned

- **Signed comparison: `slt, slti`**
- **Unsigned comparison: `sltu, sltui`**
- **Example**
  - `$s0` = 1111 1111 1111 1111 1111 1111 1111 1111
  - `$s1` = 0000 0000 0000 0000 0000 0000 0000 0001
  - `slt  $t0, $s0, $s1  # signed`
    - $-1 < +1 \Rightarrow$ `$t0 = 1`
  - `sltu $t0, $s0, $s1  # unsigned`
    - $+4{,}294{,}967{,}295 > +1 \Rightarrow$ `$t0 = 0`

Branch Instruction Design

- **Why not blt, bge, etc?**
- **Hardware for <, ≥, … slower than =, ≠**
  - Checking whether two values are equal is simpler than checking which is larger.
  - Combining comparison and branching may lengthen the critical path.
  - If the clock period increases, all instructions are affected.
- **beq and bne are the common case**
- **This is a good design compromise**

Design trade-off:
simpler branch hardware, but potentially more instructions.

Outline - II

- Supporting Procedures in Computer Hardware
- Communicating with People
- MIPS Addressing for 32-Bit Immediates and Addresses
- Translating and Starting a Program
- A Sort Example
- Arrays versus Pointers
- ARM and x86 instruction sets
- Fallacies and Pitfalls

Supporting Procedures in Computer Hardware

CHAPTER2: INSTRUCTIONS: LANGUAGE OF THE COMPUTER

Procedure Calling

- **Steps required**
  - Place parameters in registers
  - Transfer control to procedure
  - Acquire storage for procedure
  - Perform procedure's operations
  - Place result in register for caller
  - Return to place of call

Register Usage

- **$a0 – $a3: arguments (reg's 4 – 7)**
- **$v0, $v1: result values (reg's 2 and 3)**
- **$t0 – $t9: temporaries**
  - Can be overwritten by callee
- **$s0 – $s7: saved**
  - Must be saved/restored by callee
- **$gp: global pointer for static data (reg 28)**
- **$sp: stack pointer (reg 29)**
- **$fp: frame pointer (reg 30)**
- **$ra: return address (reg 31)**

Procedure Call Instructions

- **Procedure call: jump and link**
- `jal ProcedureLabel`
  - Address of following instruction put in `$ra`
  - Jumps to target address
- **Procedure return: jump register**
- `jr $ra`
  - Copies `$ra` to program counter
  - Can also be used for computed jumps
  - e.g., for case/switch statements

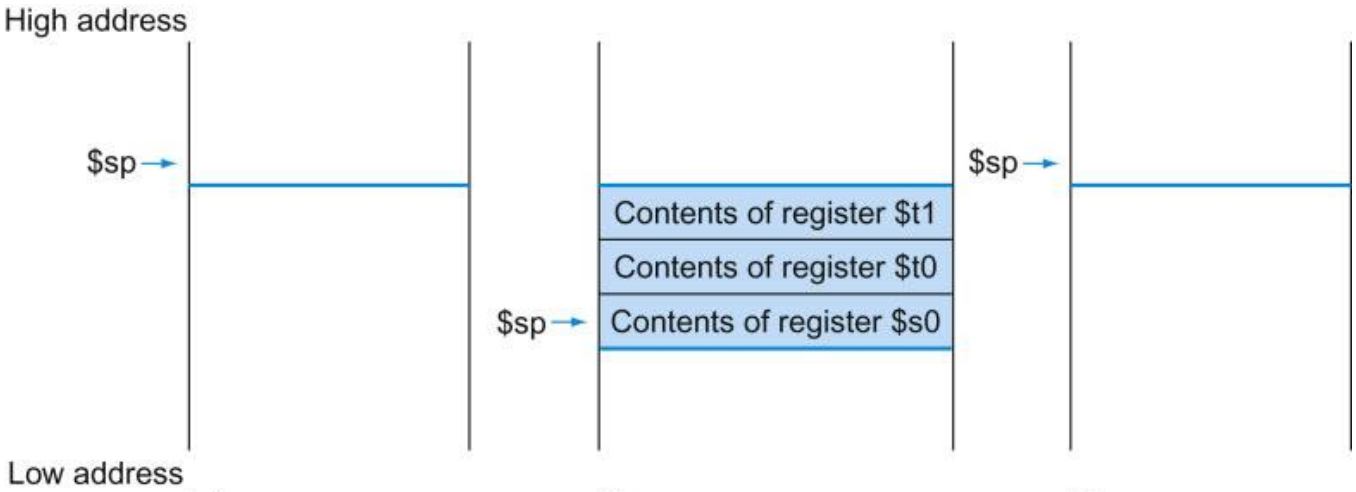
Caller

Callee

Leaf Procedure Example

- **Leaf Procedure: Procedures that do not call others**
- **C code:**
- ```
  int leaf_example (int g, h, i, j)
  { int f;
    f = (g + h) - (i + j);
    return f;
  }
  ```
  - Arguments g, h, i, j in $a0, $a1, $a2, $a3
  - f in $s0 (hence, need to **save $s0 on stack**)
  - Result in $v0
  ```

Local Data on the Stack



(a) **before** (b) **during** (c) **after**

Preserved	Not preserved
Saved registers: `$s0–$s7`	Temporary registers: `$t0–$t9`
Stack pointer register: `$sp`	Argument registers: `$a0–$a3`
Return address register: `$ra`	Return value registers: `$v0–$v1`
Stack above the stack pointer	Stack below the stack pointer

Non changeable ➔ $s0, …$s7

Leaf Procedure Example

MIPS code:

```
leaf_example:
  addi $sp, $sp, -4
  sw   $s0, 0($sp)
  add  $t0, $a0, $a1
  add  $t1, $a2, $a3
  sub  $s0, $t0, $t1
  add  $v0, $s0, $zero
  lw   $s0, 0($sp)
  addi $sp, $sp, 4
  jr   $ra
```

- ```
  int leaf_example (int g, h, i, j)
  { int f;
    f = (g + h) - (i + j);
    return f;
  }
  ```

Save $s0 on stack

Procedure body

Result

Restore $s0

Return


  ```

Non-Leaf Procedures

- **Procedures that call other procedures**
  - Nested/Non-Leaf Procedures
- **For nested call, caller needs to save on the stack:**
  - Its return address
  - Any arguments and temporaries needed after the call
- **Restore from the stack after the call**

Non-Leaf Procedure Example

- **C code:**
- ```
  int fact (int n)
  {
    if (n < 1) return 1;
    else return n * fact(n - 1);
  }
  ```
  - Argument n in $a0
  - Result in $v0
  ```

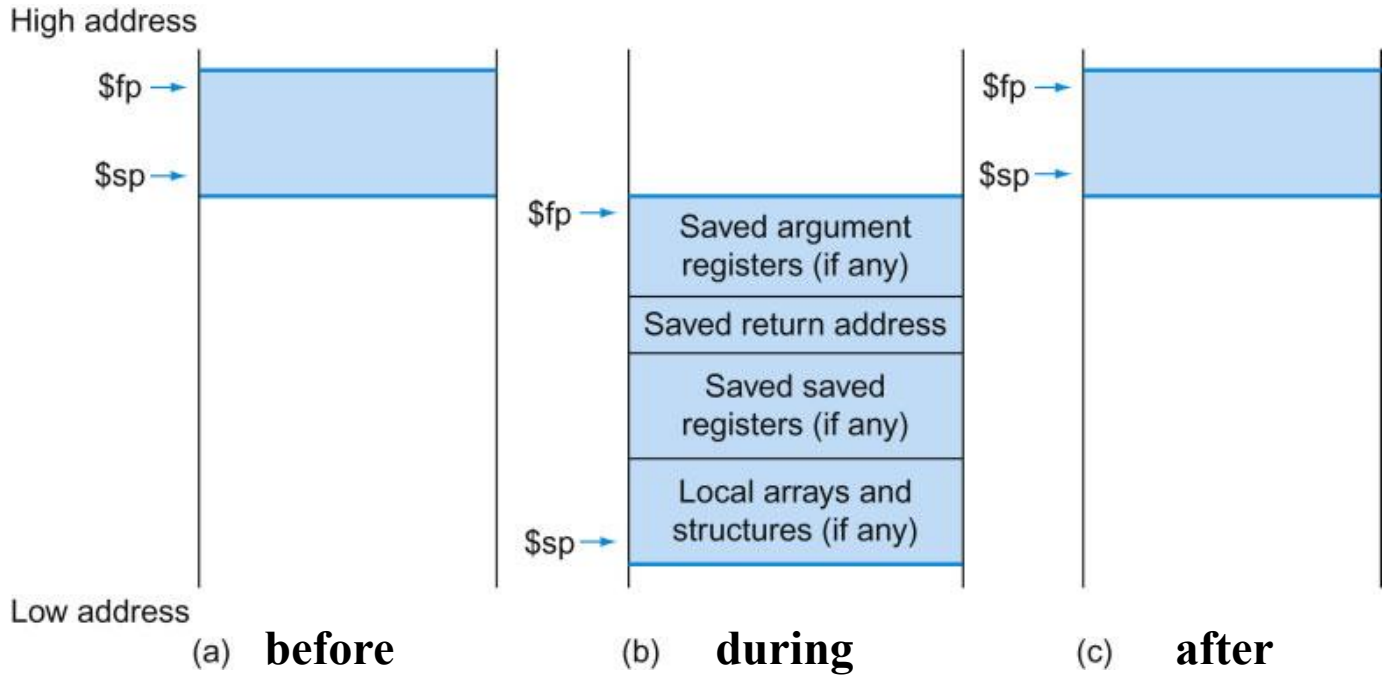
Non-Leaf Procedure Example

- ```
  int fact (int n)
  {
    if (n < 1) return 1;
    else return n * fact(n- 1);
  }
  ```

MIPS code:

```
fact:
    addi $sp, $sp, -8       # adjust stack for 2 items
    sw   $ra, 4($sp)        # save return address
    sw   $a0, 0($sp)        # save argument
    slti $t0, $a0, 1        # test for n < 1
    beq  $t0, $zero, L1
    addi $v0, $zero, 1      # if so, result is 1
    addi $sp, $sp, 8        #   pop 2 items from stack
    jr   $ra                #   and return
L1: addi $a0, $a0, -1       # else decrement n
    jal  fact               # recursive call
R:  lw   $a0, 0($sp)        # restore original n
    lw   $ra, 4($sp)        #   and return address
    addi $sp, $sp, 8        # pop 2 items from stack
    mul  $v0, $a0, $v0      # multiply to get result
    jr   $ra                # and return
```
```

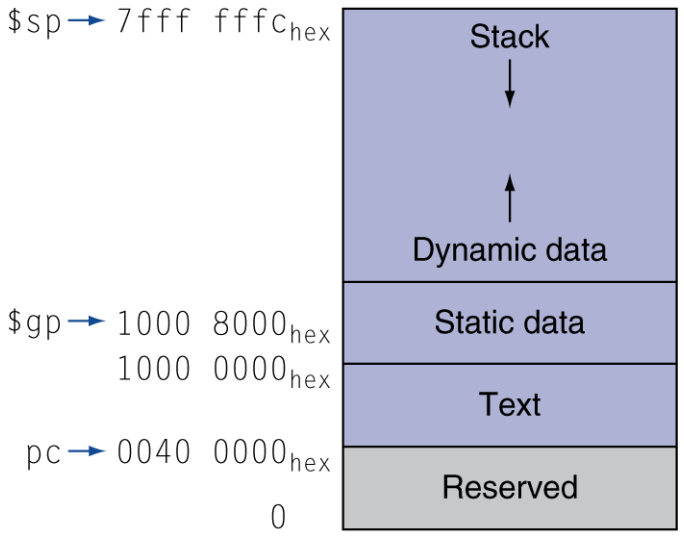
Local Data on the Stack



- **Local data allocated by callee**
  - e.g., C automatic variables
- **Procedure frame (activation record)**
  - Used by some compilers to manage stack storage

Memory Layout

- **Text: program code**
- **Static data: global variables**
  - e.g., static variables in C, constant arrays and strings
  - $gp initialized to address allowing ±offsets into this segment
- **Dynamic data: heap**
  - E.g., malloc in C, new in Java
- **Stack: automatic storage**



Communicating with People

CHAPTER2: INSTRUCTIONS: LANGUAGE OF THE COMPUTER

Character Data

- **Byte-encoded character sets**
  - ASCII: 128 characters
    - 95 graphic, 33 control
  - Latin-1: 256 characters
    - ASCII, +96 more graphic characters
- **Unicode: 32-bit character set**
  - Used in Java, C++ wide characters, …
  - Most of the world's alphabets, plus symbols
  - UTF-8, UTF-16: variable-length encodings

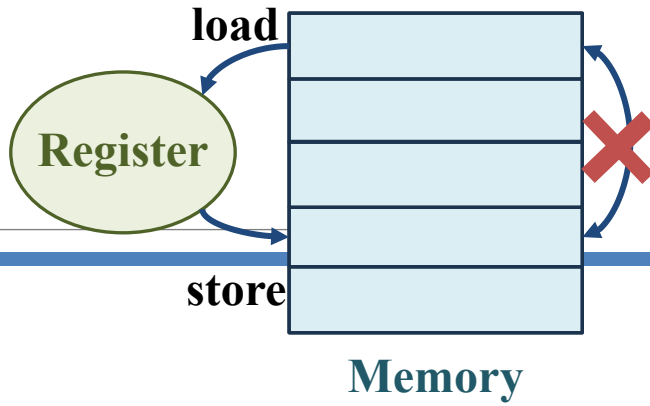
Byte/Halfword Operations

- **Could use bitwise operations**
- **MIPS byte/halfword load/store**
  - String processing is a common case
- **lb rt, offset(rs) lh rt, offset(rs)**
  - Sign extend to 32 bits in rt
- **lbu rt, offset(rs) lhu rt, offset(rs)**
  - Zero extend to 32 bits in rt
- **sb rt, offset(rs) sh rt, offset(rs)**
  - Store just **rightmost** byte/halfword

String Copy Example

- **C code (naïve):**
  - Null-terminated string
- ```
  void strcpy (char x[], char y[])
  { int i;
    i = 0;
    while ((x[i]=y[i])!='\0')
      i += 1;
  }
  ```
  - Addresses of x, y in $a0, $a1
  - i in $s0
  ```

String Copy Example



MIPS code:

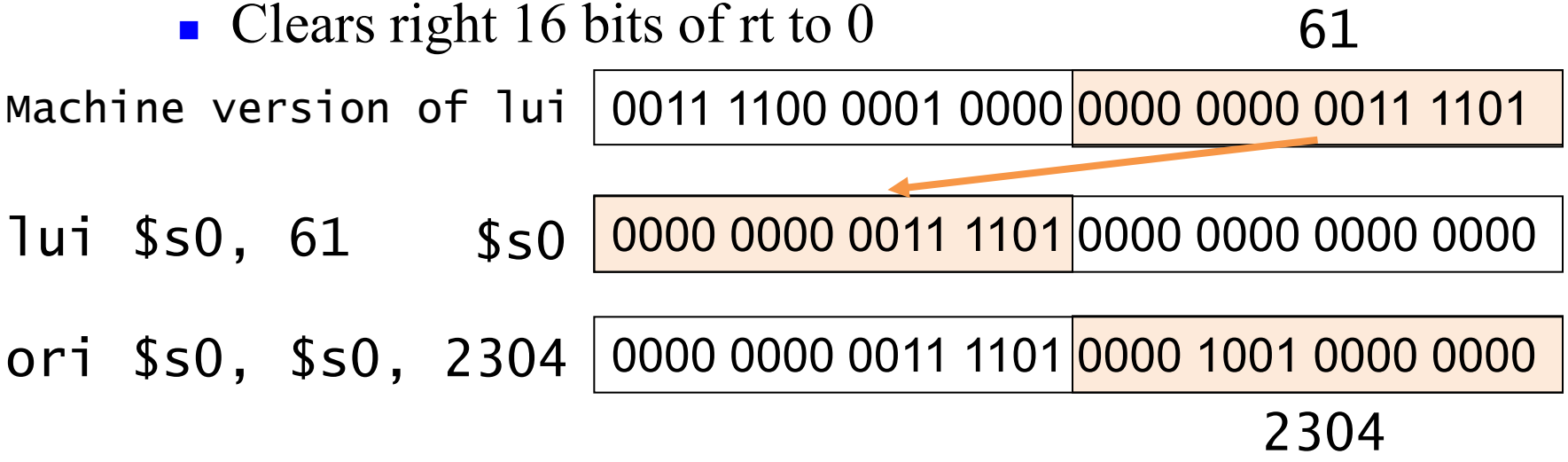
```
strcpy:
    addi $sp, $sp, -4       # adjust stack for 1 item
    sw   $s0, 0($sp)        # save $s0
    add  $s0, $zero, $zero  # i = 0
L1: add  $t1, $s0, $a1      # addr of y[i] in $t1
    lbu  $t2, 0($t1)        # $t2 = y[i]
    add  $t3, $s0, $a0      # addr of x[i] in $t3
    sb   $t2, 0($t3)        # x[i] = y[i]
    beq  $t2, $zero, L2     # exit loop if y[i] == 0
    addi $s0, $s0, 1        # i = i + 1
    j    L1                 # next iteration of loop
L2: lw   $s0, 0($sp)        # restore saved $s0
    addi $sp, $sp, 4        # pop 1 item from stack
    jr   $ra                # and return
```

MIPS Addressing for 32-Bit Immediates and Addresses

CHAPTER2: INSTRUCTIONS: LANGUAGE OF THE COMPUTER

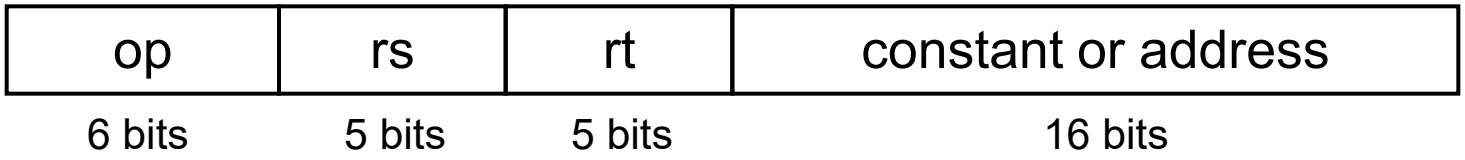
32-bit Constants

- **Most constants are small**
  - 16-bit immediate is sufficient
- **For the occasional 32-bit constant**
- **lui rt, constant** **# Load upper immediate (lui)**
  - Copies 16-bit constant to left 16 bits of rt
  - Clears right 16 bits of rt to 0



Branch Addressing

- **Branch instructions specify**
  - Opcode, two registers, target address
- **Most branch targets are near branch**
  - Forward or backward

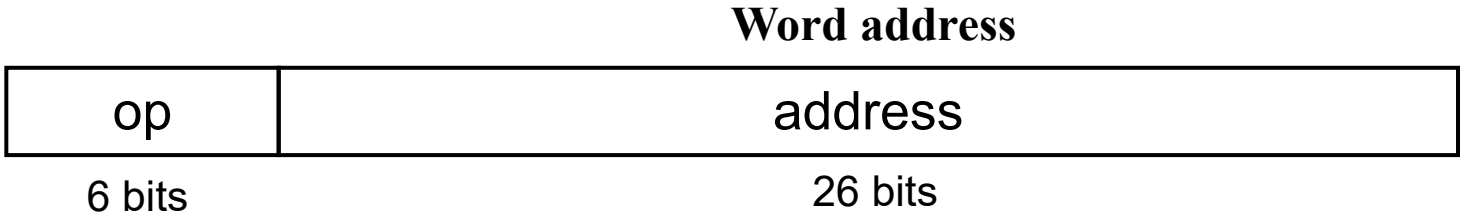


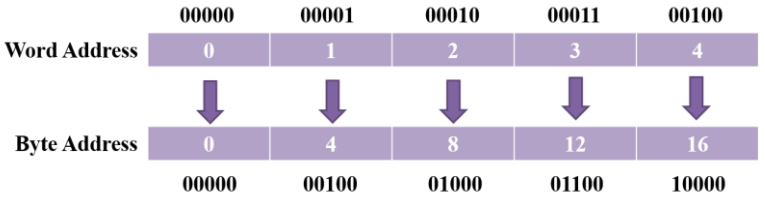
- **PC(Program Counter)-relative addressing**
  - Target address = PC + offset × 4
  - PC already incremented by 4 by this time

Each instruction is 32-bit

Jump Addressing

- **Jump (j and jal) targets could be anywhere in text segment**
  - Encode pseudo-direct address in instruction





- **(Pseudo)Direct jump addressing**
  - Target address = PC31…28 : (address × 4)

PC is 32-bits
Leaving **the upper 4-bit** of PC

Avoid to place a program across the address boundary

Target Addressing Example

- **Loop code from earlier example**
  - Assume Loop at location 80000

```
Loop: sll  $t1, $s3, 2
      add  $t1, $t1, $s6
      lw   $t0, 0($t1)
      bne  $t0, $s5, Exit
      addi $s3, $s3, 1
      j    Loop
Exit: …
```

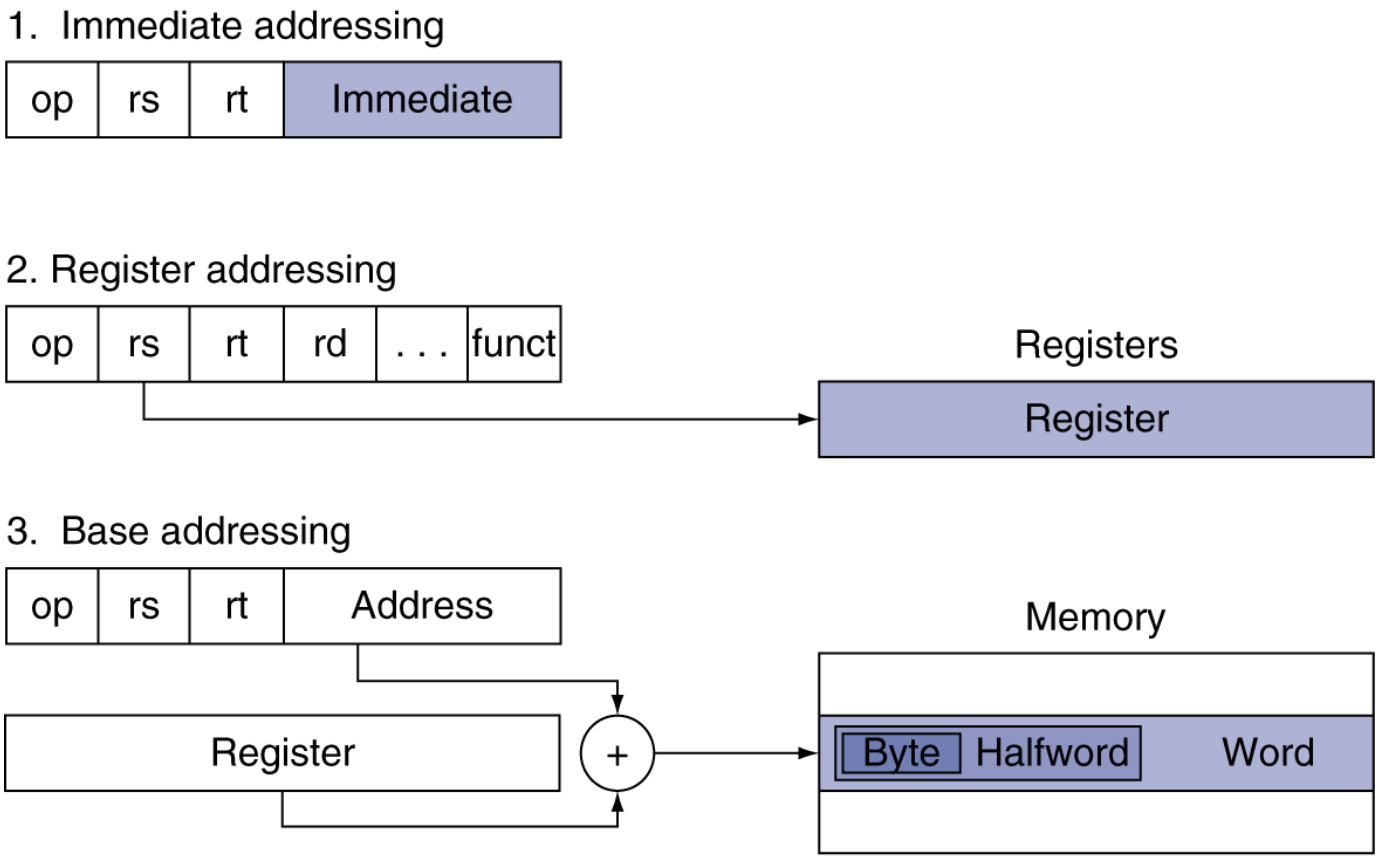
80000	0	0	19	9	4	0
80004	0	9	22	9	0	32
80008	35	9	8	0		
80012	5	8	21	2		
80016	8	19	19	1		
80020	2	20000				
80024						

Branching Far Away

- **If branch target is too far to encode with 16-bit offset, assembler rewrites the code**
- **Example**

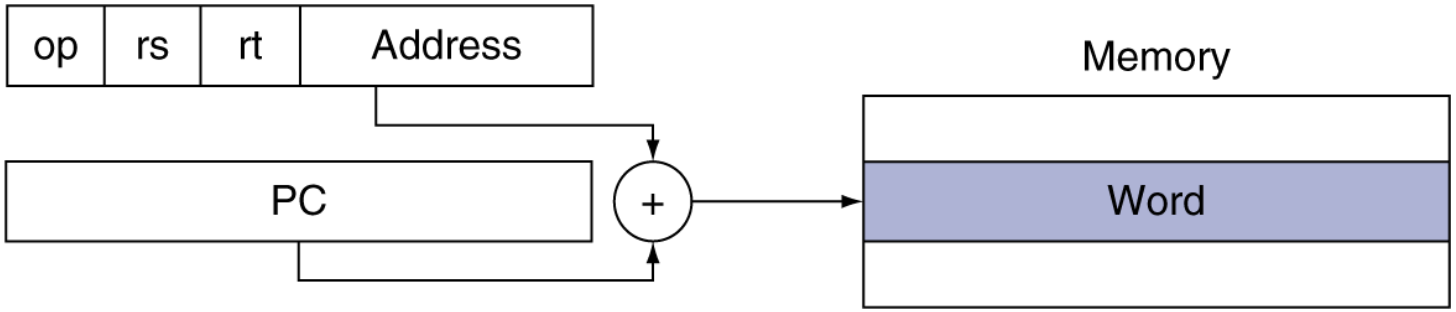
```
beq $s0,$s1, L1
        ↓
bne $s0,$s1, L2
   j L1
L2:  …
```

Addressing Mode Summary

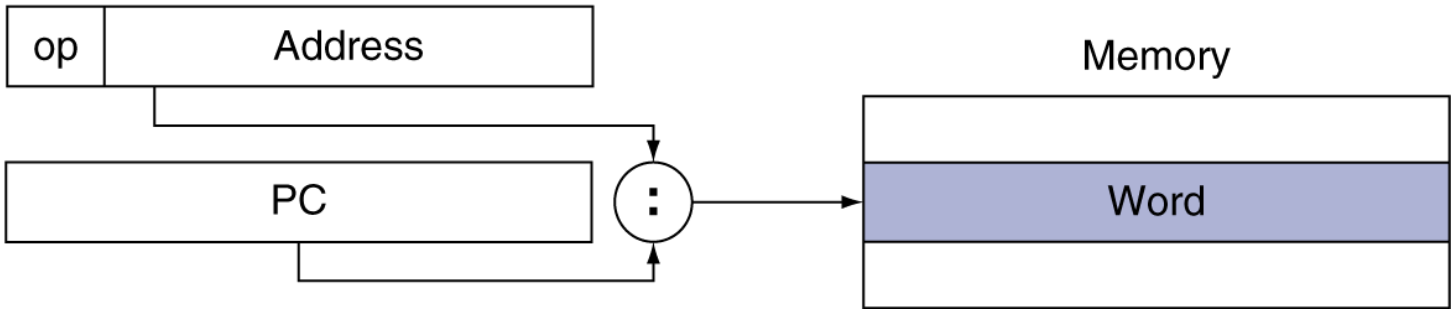


Addressing Mode Summary

4. PC-relative addressing



5. Pseudodirect addressing





Slido Q&A

Review of last week…

CHAPTER2: INSTRUCTIONS: LANGUAGE OF THE COMPUTER

Why do we need instructions?

- Machine cannot understand the natural language Sometimes, human couldn't either…
- Design specific instructions ➔ ISA
  - Easily communicate with CPU to operate with data
- Operations (in MIPS)
  - Arithmetic
  - Data Transfer
  - Logical
  - Conditional
- MIPS has three formats to handle different operations

CPU only has 32 Registers

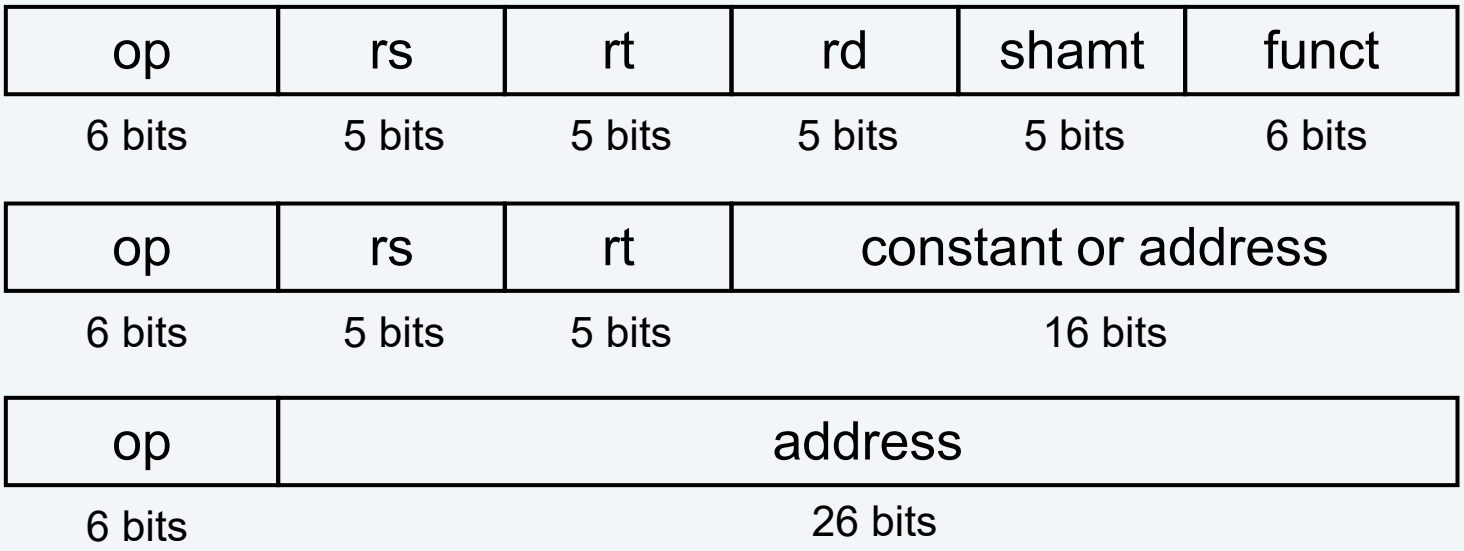
Stored data in Memory ➔ Locate the specific data

Before use data, you have to Load it to Register
After use it, you have to Store it back to Memory

Representing Instructions

Three Instruction Formats

R-type **I-type** **J-type**



Addressing Mode Summary

Example

1. Immediate addressing



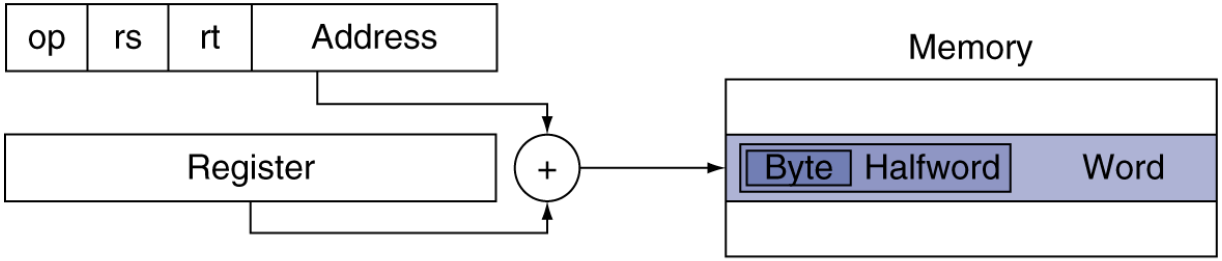

```
   addi $s1,$s2,20
➔  $s1 = $s2 + 20
```

2. Register addressing




```
   add $s1,$s2,$s3
➔  $s1 = $s2 + $s3
```

3. Base addressing




```
   lw $s1, 20($s2)
➔  read 4 bytes from Mem[$s2+20]
```

```
   lh $s1, 20($s2)
➔  read 2 bytes from Mem[$s2+20]
```

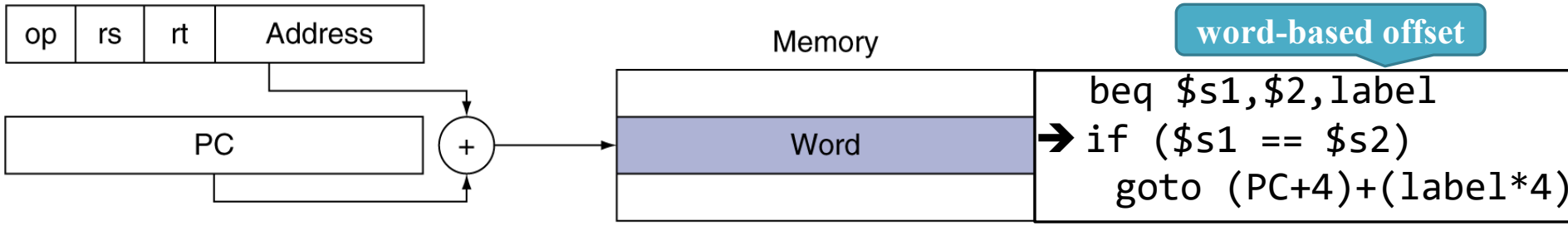
```
   lb $s1, 20($s2)
➔  read 1 byte from Mem[$s2+20]
```

Assign register to acquire data in memory

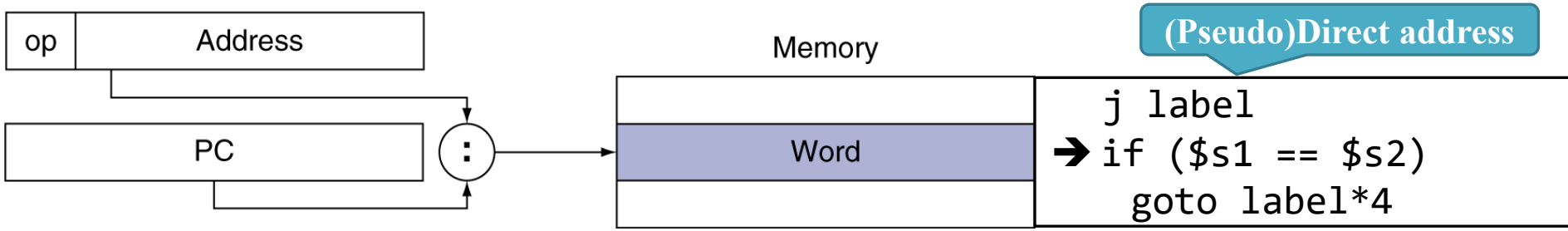
Addressing Mode Summary

Example

4. PC-relative addressing



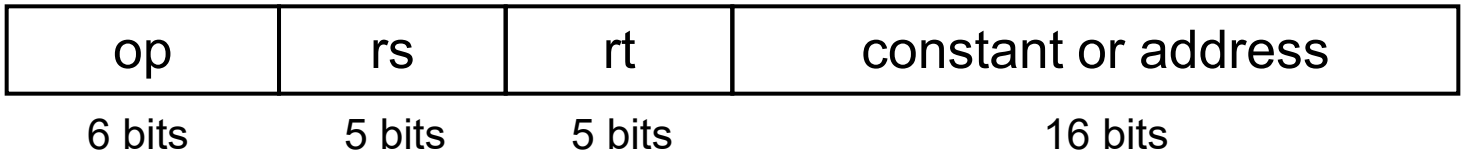
5. Pseudodirect addressing



Assign PC to specific code address in memory

Branch Addressing

- **Branch instructions specify**
  - Opcode, two registers, target address
- **Most branch targets are near branch**
  - Forward or backward

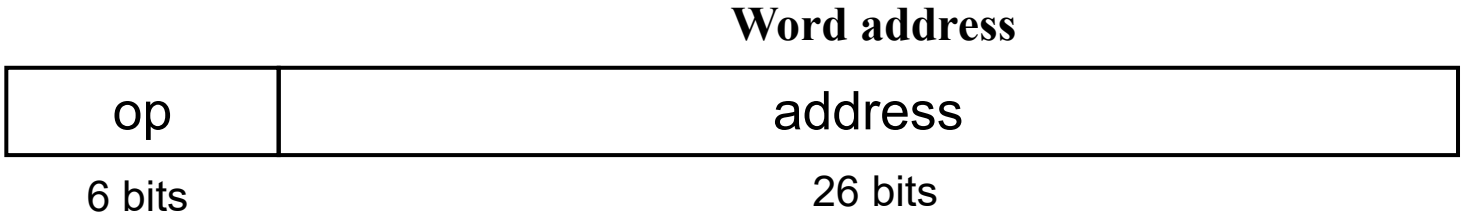


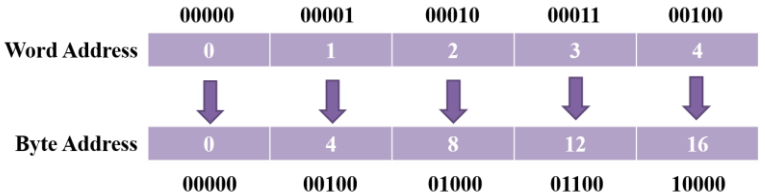
- **PC(Program Counter)-relative addressing**
  - Target address = PC + offset × 4
  - PC already incremented by 4 by this time

Each instruction is 32-bit

Jump Addressing

- **Jump (j and jal) targets could be anywhere in text segment**
  - Encode full address in instruction





- **(Pseudo)Direct jump addressing**
  - Target address = PC31…28 : (address × 4)

PC is 32-bits
Leaving **the upper 4-bit** of PC

Avoid to place a program across the address boundary

Target Addressing Example

- **Loop code from earlier example**
  - Assume Loop at location 80000

```
Loop: sll  $t1, $s3, 2
      add  $t1, $t1, $s6
      lw   $t0, 0($t1)
      bne  $t0, $s5, Exit
      addi $s3, $s3, 1
      j    Loop
Exit: …
```

80000	0	0	19	9	4	0
80004	0	9	22	9	0	32
80008	35	9	8	0		
80012	5	8	21	2		
80016	8	19	19	1		
80020	2	20000				
80024						

Branching Far Away

- **If branch target is too far to encode with 16-bit offset, assembler rewrites the code**
- **Example**

```
beq $s0,$s1, L1
```

inverse ↓

```
bne $s0,$s1, L2
   j L1
L2:  …
```

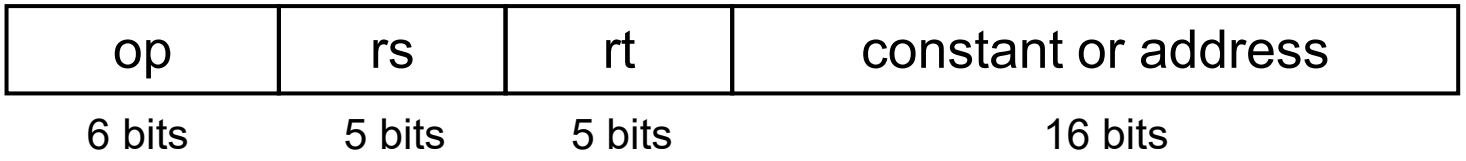
```
if ($s0 == $s1) go to L1
```

```
if ($s0 != $s1) go to L2
else go to L1
```

Apply J-type to increase address length

16 bit << address 32-bit

I-type



Operations

Arithmetic

add $s0, $t1, $t2 — R-type

addi $s0, $t1, 20 — I-type

sub $s0, $t1, $t2 — R-type

Logical

sll $t1, $t2, 2

and $t0, $t1, $t2

or $t0, $t1, $t2

nor $t0, $t1, $t2

R-type

Data Transfer

lw $t0, 8($s1)

sw $t0, 8($s1)

lb $t0, 3($s1)

lui $s0, 61

I-type

Conditional

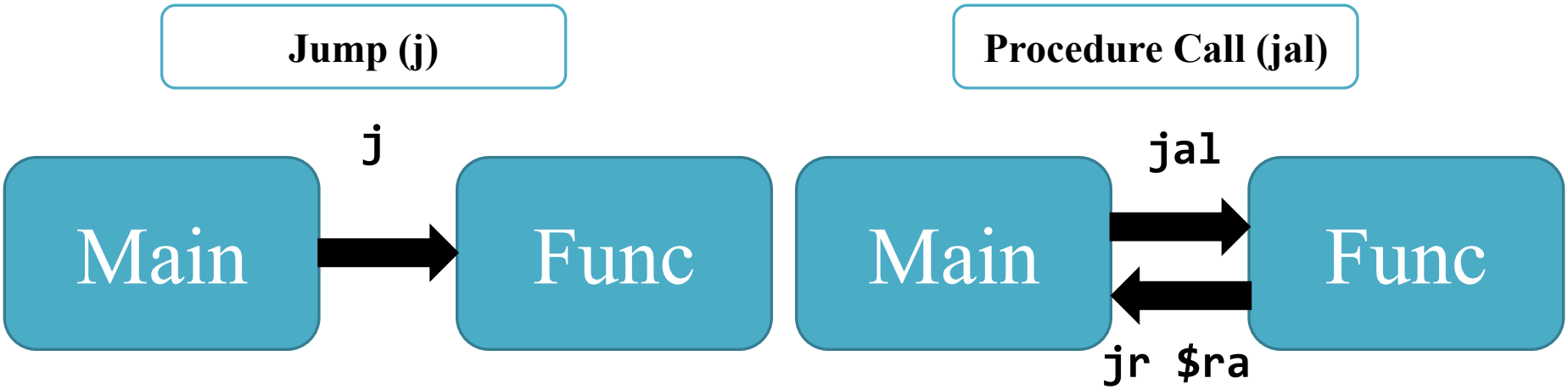
bne $t0, $t1, Label — I-type

beq $t0, $t1, Label — I-type

slt $t0, $s1, $s2 — R-type

slti $t0, $s1, 10 — I-type

From Jump Instructions to Procedure Calls



How can we jump to a function and then return?

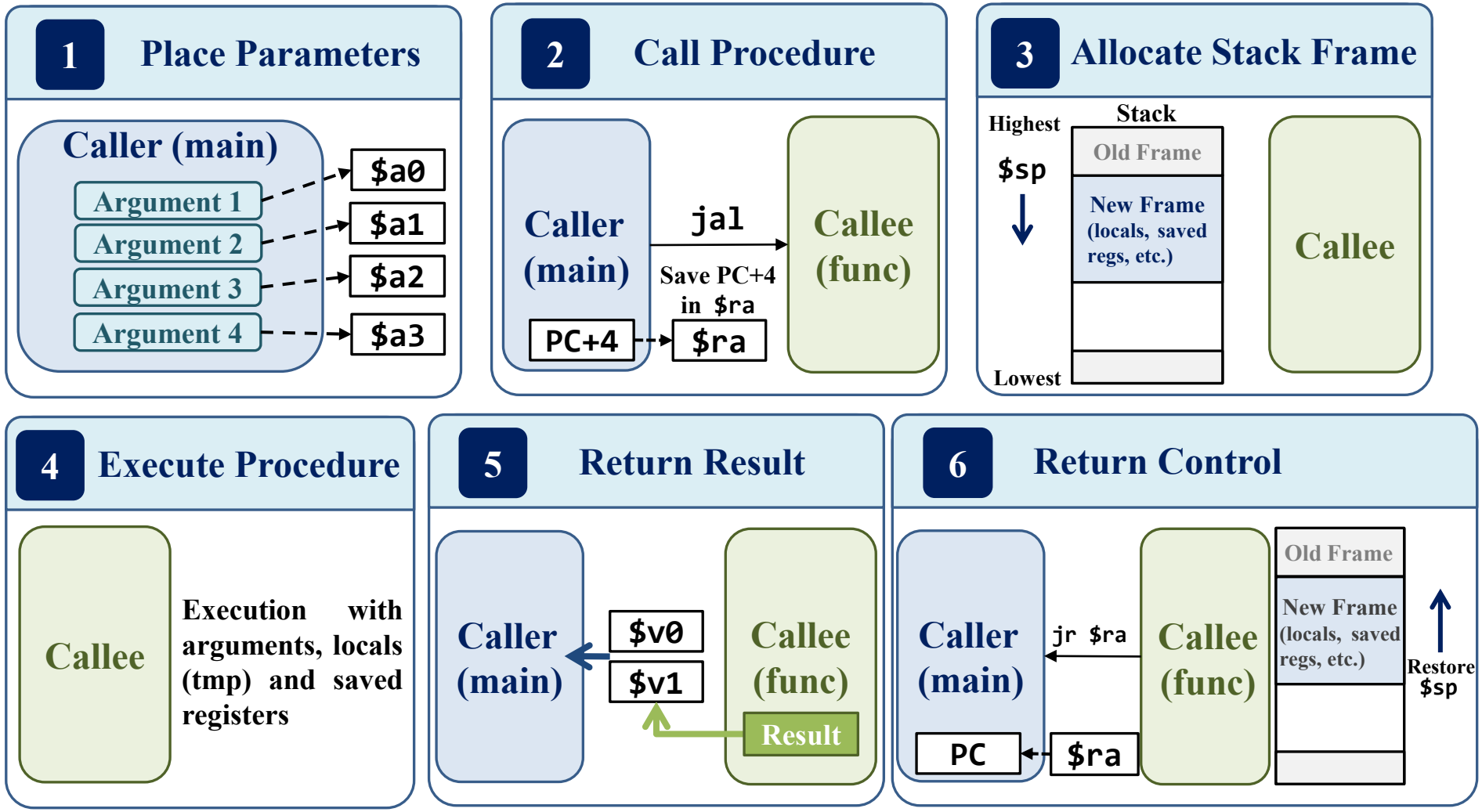
Procedure Call Instructions

- **Procedure call: jump and link**
- `jal ProcedureLabel`
  - 1. Address of following instruction put in `$ra`
  - 2. Jumps to target address
- **Procedure return: jump register**
- `jr $ra`
  - Copies `$ra` to program counter
  - Can also be used for computed jumps
  - e.g., for case/switch statements

Caller

Callee

Procedure Calling



MIPS Register Conventions

Name	Register number	Usage	Preserved on call?
$zero	0	The constant value 0	n.a.
$v0–$v1	2–3	Values for results and expression evaluation	no
$a0–$a3	4–7	Arguments	no
$t0–$t7	8–15	Temporaries	no
$s0–$s7	16–23	Saved	yes
$t8–$t9	24–25	More temporaries	no
$gp	28	Global pointer	yes
$sp	29	Stack pointer	yes
$fp	30	Frame pointer	yes
$ra	31	Return address	yes

Caller vs. Callee Responsibilities

Caller's preservation responsibility to use S registers

Callee's preservation responsibility to use VAT registers

Caller (Main)

- Place arguments in `$a0 - $a3`
- `jal` saves return address in `$ra`.
- Save any **caller-saved registers** if needed

  `(VAT: $v0-$v1,$a0-$a3,$t0-$t9)`
- After the call, read the return value from `$v0-$v1`

Callee (func)

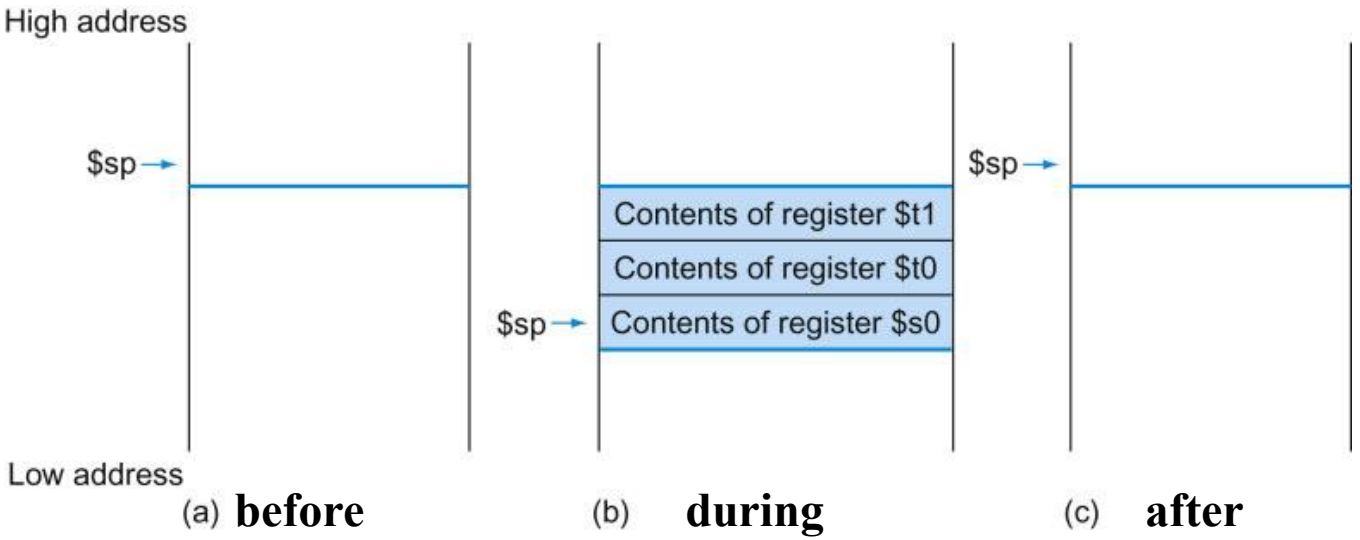
- Allocate stack frame if needed
- Save and restore **callee-saved registers** that are modified

  `(saved register: $s0-$s7)`
- Execute procedure
- Place return value in `$v0-$v1`
- Restore stack pointer before returning with `jr $ra`

Leaf Procedure Example

- **Leaf Procedure: Procedures that do not call others**
- **C code:**
- ```
  int leaf_example (int g, h, i, j)
  { int f;
    f = (g + h) - (i + j);
    return f;
  }
  ```
  - Arguments g, h, i, j in $a0, $a1, $a2, $a3
  - f in $s0 (hence, need to **save $s0 on stack**)
  - Result in $v0
  ```

Local Data on the Stack



Preserved	Not preserved
Saved registers: `$s0–$s7`	Temporary registers: `$t0–$t9`
Stack pointer register: `$sp`	Argument registers: `$a0–$a3`
Return address register: `$ra`	Return value registers: `$v0–$v1`
Stack above the stack pointer	Stack below the stack pointer

Leaf Procedure Example

MIPS code:

```
leaf_example:
  addi $sp, $sp, -4
  sw   $s0, 0($sp)
  add  $t0, $a0, $a1
  add  $t1, $a2, $a3
  sub  $s0, $t0, $t1
  add  $v0, $s0, $zero
  lw   $s0, 0($sp)
  addi $sp, $sp, 4
  jr   $ra
```

-
```
int leaf_example (int g, h, i, j)
{ int f;
  f = (g + h) - (i + j);
  return f;
}
```

Save $s0 on stack

Procedure body

Result

Restore $s0

Return

Leaf Procedure Example

```
int leaf_example (int g, h, i, j)
{ int f;
  f = (g + h) - (i + j);
  return f;
}
```

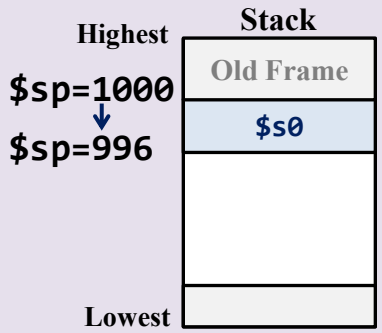
$a0 $a1 $a2 $a3 $s0

MIPS code:

leaf_example:	
addi $sp, $sp, -4	Save $s0 on stack
sw $s0, 0($sp)	
add $t0, $a0, $a1	Procedure body
add $t1, $a2, $a3	
sub $s0, $t0, $t1	
add $v0, $s0, $zero	Result
lw $s0, 0($sp)	Restore $s0
addi $sp, $sp, 4	
jr $ra	Return

1 Save $s0 on stack

```
$sp = $sp - 4
mem[$sp+0] = $s0
```

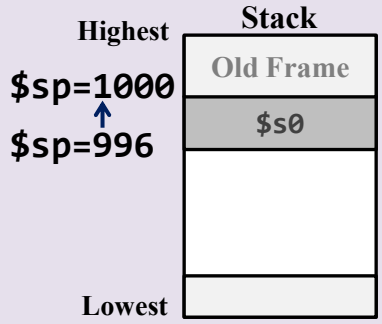


2 Execute Procedure

```
$t0 = g+h
$t1 = i+j
$s0 = $t0 - $t1
Return $s0 to $v0
```

3 Restore $s0 on stack

```
$s0 = mem[$sp+0]
$sp = $sp + 4
```



Non-Leaf Procedures

- **Procedures that call other procedures**
  - Nested/Non-Leaf Procedures
- **For nested call, caller needs to save on the stack:**
  - Its return address
  - Any arguments and temporaries needed after the call
- **Restore from the stack after the call**

Preserved	Not preserved
Saved registers: `$s0–$s7`	Temporary registers: `$t0–$t9`
Stack pointer register: `$sp`	Argument registers: `$a0–$a3`
Return address register: `$ra`	Return value registers: `$v0–$v1`
Stack above the stack pointer	Stack below the stack pointer

Non-Leaf Procedure Example

- **C code:**
- ```
  int fact (int n)
  {
    if (n < 1) return 1;
    else return n * fact(n - 1);
  }
  ```
  - Argument n in $a0
  - Result in $v0

n! = n*(n-1)*(n-2)*(n-3)*…*2*1*1
  ```

Non-Leaf Procedure Example

- ```
  int fact (int n)
  {
    if (n < 1) return 1;
    else return n * fact(n- 1);
  }
  ```

MIPS code:

```
fact:
    addi $sp, $sp, -8       # adjust stack for 2 items
    sw   $ra, 4($sp)        # save return address
    sw   $a0, 0($sp)        # save argument
    slti $t0, $a0, 1        # test for n < 1
    beq  $t0, $zero, L1
    addi $v0, $zero, 1      # if so, result is 1
    addi $sp, $sp, 8        #   pop 2 items from stack
    jr   $ra                #   and return
L1: addi $a0, $a0, -1       # else decrement n
    jal  fact               # recursive call
R:  lw   $a0, 0($sp)        # restore original n
    lw   $ra, 4($sp)        #   and return address
    addi $sp, $sp, 8        # pop 2 items from stack
    mul  $v0, $a0, $v0      # multiply to get result
    jr   $ra                # and return
```
```

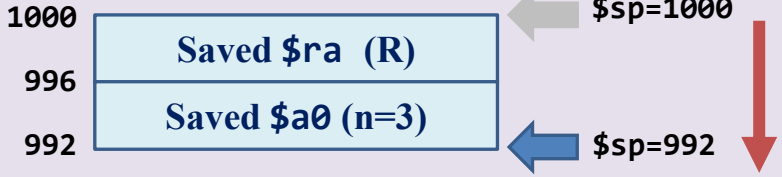
```
int fact(int n){
      if(n < 1) return 1;
      else return n * fact(n - 1);
}
```

Non-Leaf Procedure Example
fact(3)

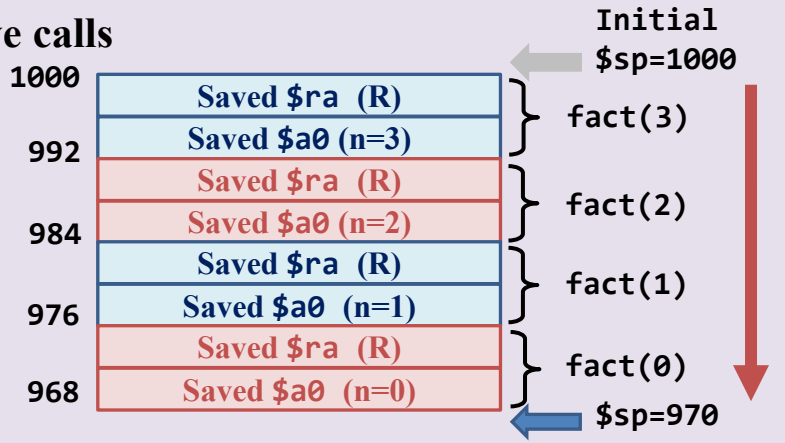
```
fact:  addi $sp, $sp, -8
       sw $ra, 4($sp)
       sw $a0, 0($sp)
       slti $t0, $a0, 1
       beq $t0, $zero, L1
       addi $v0, $zero, 1
       addi $sp, $sp, 8
       jr  $ra
L1:    addi $a0, $a0, -1
       jal  fact
R:     lw $a0, 0($sp)
       lw $ra, 4($sp)
       addi $sp, $sp, 8
       mul $v0, $a0, $v0
       jr $ra
```

Why must save $ra and $a0 here?

1 **Save $ra and $a0**



2 **Recursive calls**

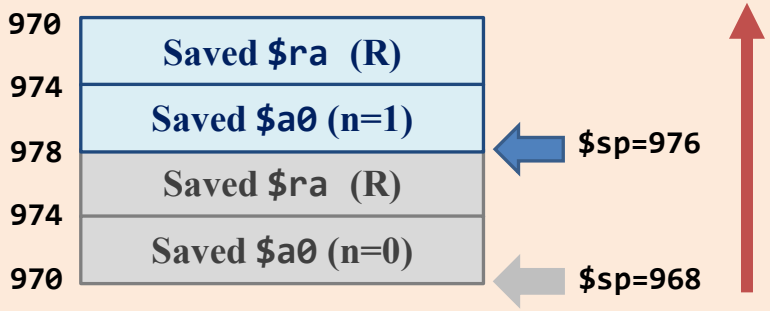


```
int fact(int n){
      if(n < 1) return 1;
      else return n * fact(n - 1);
}
```

Non-Leaf Procedure Example
fact(3)

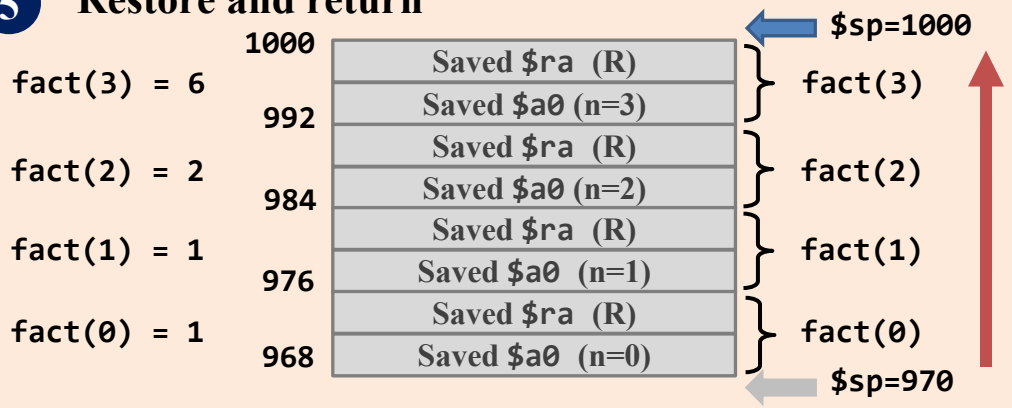
```
fact:  addi $sp, $sp, -8
       sw $ra, 4($sp)
       sw $a0, 0($sp)
       slti $t0, $a0, 1
       beq $t0, $zero, L1
fact(0)
       addi $v0, $zero, 1
       addi $sp, $sp, 8
       jr  $ra
L1:    addi $a0, $a0, -1
       jal  fact
R:     lw $a0, 0($sp)
       lw $ra, 4($sp)
fact(1)
       addi $sp, $sp, 8
       mul $v0, $a0, $v0
       jr $ra
```

3 Restore $sp and $ra



4 Execute multiplication, store in $v0 and return to $ra

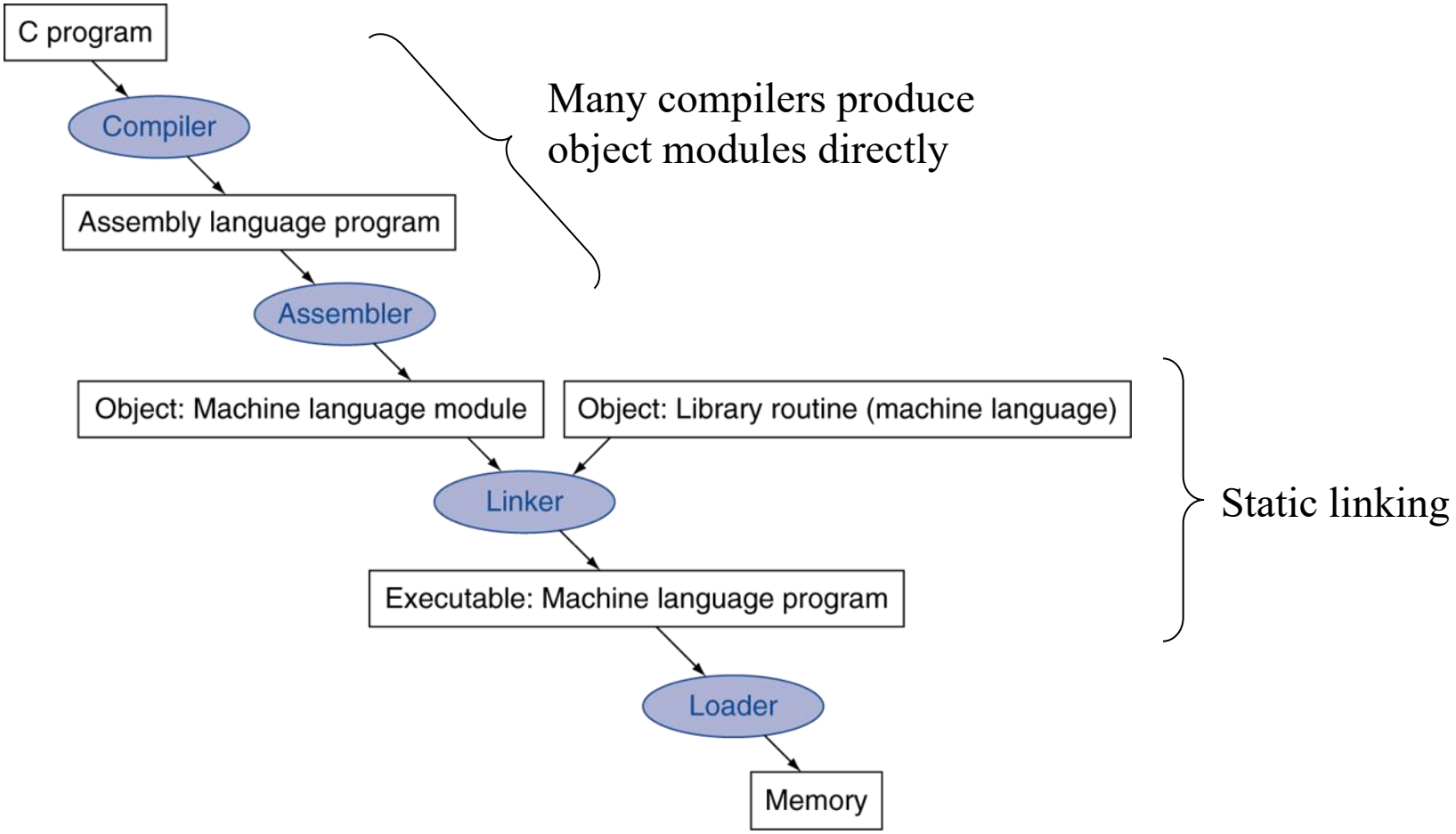
5 Restore and return



Translating and Starting a Program

CHAPTER2: INSTRUCTIONS: LANGUAGE OF THE COMPUTER

Translation and Startup



Translation and Startup

```
int leaf_example (int g, h, i, j)
{ int f;
  f = (g + h) - (i + j);
  return f;
}
```

How is the program prepared to run?

Executable file

Loader

Compiler

Linker

Process Memory

What instructions are needed?

How are programs connected

```
leaf_example:
  addi $sp, $sp, -4
  sw   $s0, 0($sp)
  add  $t0, $a0, $a1
  add  $t1, $a2, $a3
  sub  $s0, $t0, $t1
  add  $v0, $s0, $zero
  lw   $s0, 0($sp)
  addi $sp, $sp, 4
  jr   $ra
```

```
int add(int, int);
int main(){
    return add(2,3);
}
```

```
int add(int a, int b){
    return a + b;
}
```

Assembler

op	rs	rt	constant or address
6 bits	5 bits	5 bits	16 bits
001000	10010	10001	0000 0000 0110 0100

How are instructions encoded?

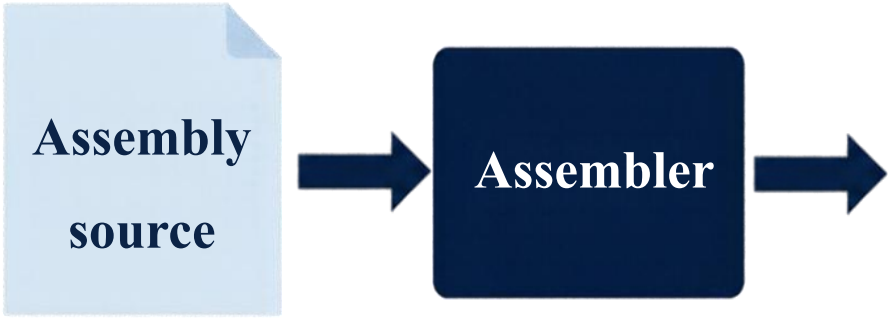
Assembler Pseudo-instructions

- Most assembler instructions represent machine instructions one-to-one
- A **pseudo-instruction** is a convenient assembly instruction that the assembler **expands into one or more** real machine instructions.

Pseudo-instruction	CPU Executed ISA
`move $t0, $t1`	`add $t0, $zero, $t1`
`blt $t0, $t1, L`	`slt $at, $t0, $t1` `bne $at, $zero, L`

- **General Purpose Register:** `$at` (register 1)
- `$at` uses for assembler temporary data

Producing an Object Module



Object code includes machine instructions

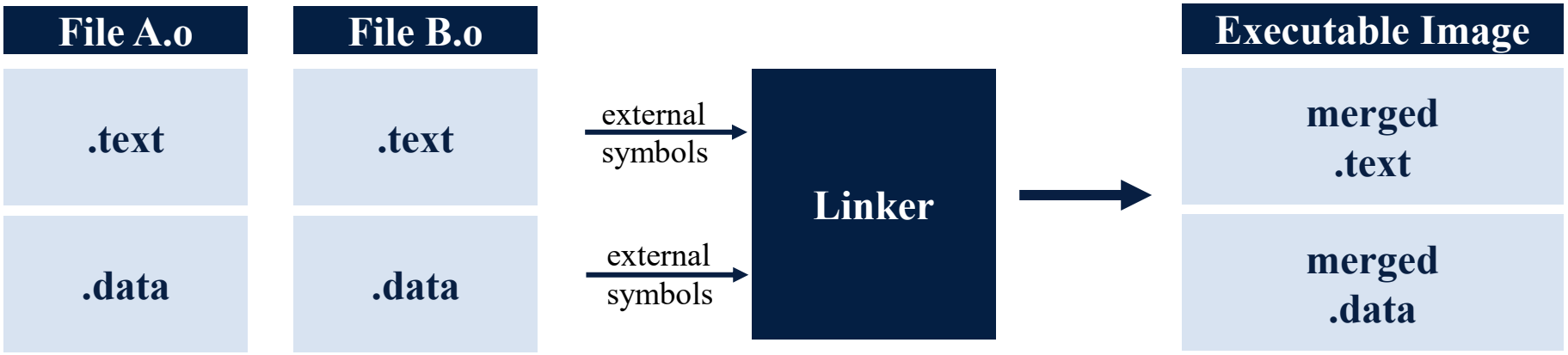
AND information required for linking.

An object module is not an executable.

Object module (.o)

Header	section information
Text	.text machine instructions
Static Data	.data program-lifetime data
Relocation entries	locations needing address fixes
Symbol table	definitions and external references
Debug information	source mappings

Linking Object Modules



1 Merge sections

Combine text and data from all object files.

2 Resolve symbols

Connect references, e.g., call in A to function in B.

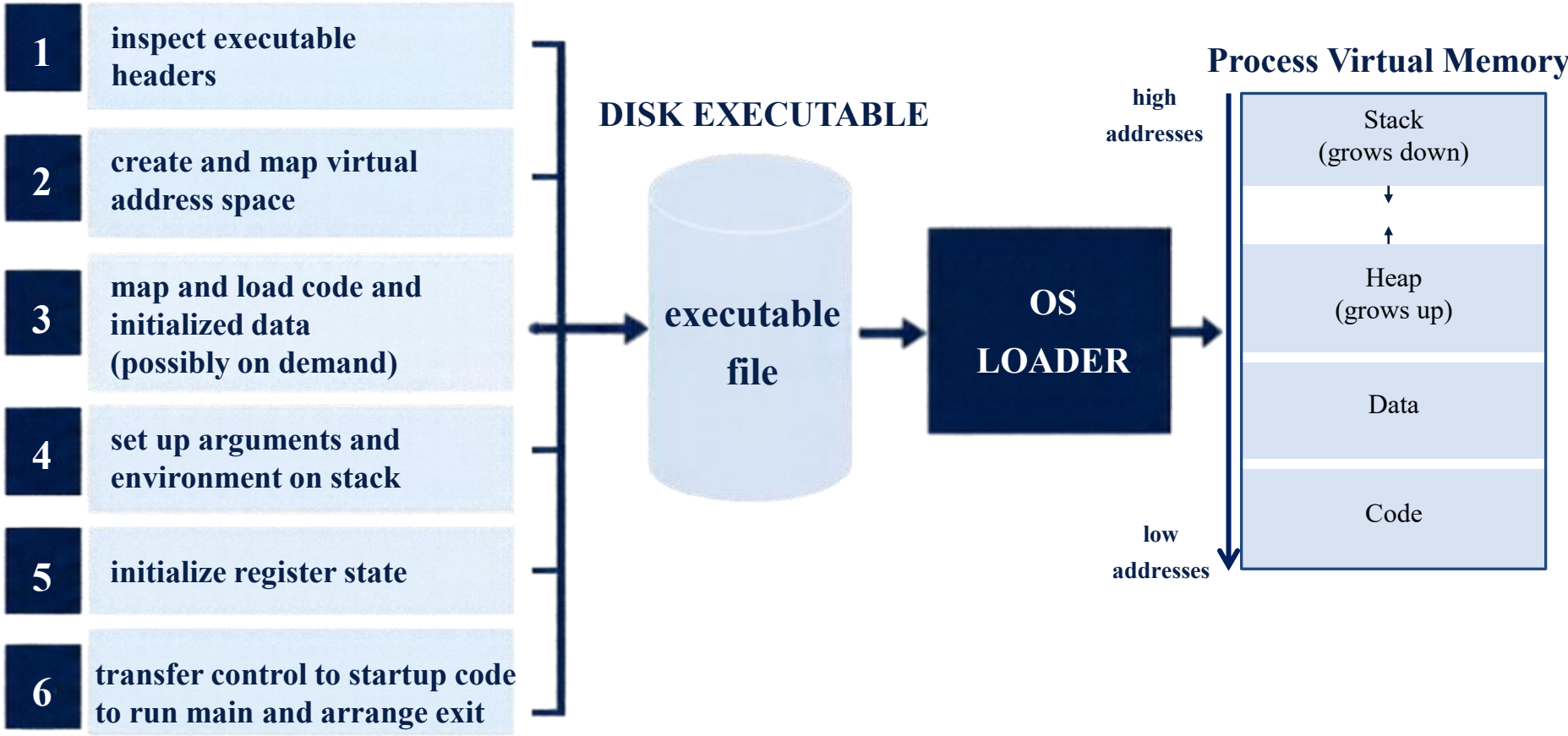
3 Apply relocations

Patch location-dependent references with final addresses.

Some relocations may remain for the runtime loader or dynamic linker to resolve.

Loading a Program

From executable on disk to a running program in memory.

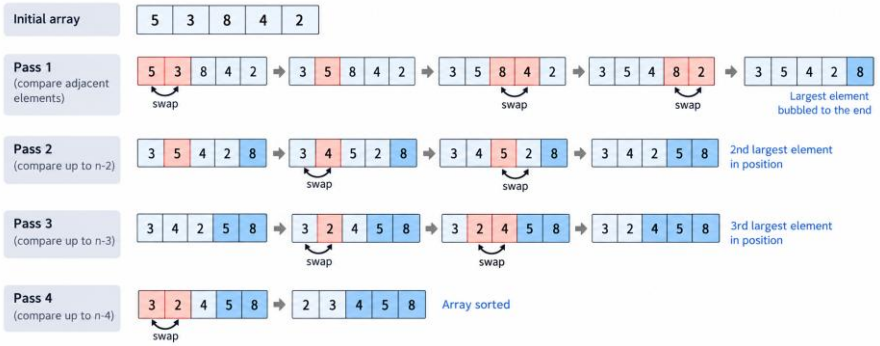


A Sort Example

CHAPTER2: INSTRUCTIONS: LANGUAGE OF THE COMPUTER

C Sort Example

- **Illustrates use of assembly instructions for a C bubble sort function**
- **Swap procedure (leaf)**
  - ```
    void swap(int v[], int k)
    {
      int temp;
      temp = v[k];
      v[k] = v[k+1];
      v[k+1] = temp;
    }
    ```
  - v in $a0, k in $a1, temp in $t0


    ```

The Procedure Swap

```
swap: sll $t1, $a1, 2    # $t1 = k * 4
      add $t1, $a0, $t1  # $t1 = v+(k*4)
                         #   (address of v[k])
      lw $t0, 0($t1)     # $t0 (temp) = v[k]
      lw $t2, 4($t1)     # $t2 = v[k+1]
      sw $t2, 0($t1)     # v[k] = $t2 (v[k+1])
      sw $t0, 4($t1)     # v[k+1] = $t0 (temp)
      jr $ra             # return to calling routine
```

The Sort Procedure in C

- **Non-leaf (calls swap)**

```
void sort (int v[], int n)
{
  int i, j;
  for (i = 0; i < n; i += 1) {
    for (j = i - 1;
         j >= 0 && v[j] > v[j + 1];
         j -= 1) {
      swap(v,j);
    }
  }
}
```

v in $a0, k in $a1, i in $s0, j in $s1

How to implement for loop?

1. Initialized of i

2. Conditional branch with i

3. Incremented of i

The Procedure Body

```
        move $s2, $a0           # save $a0 into $s2
        move $s3, $a1           # save $a1 into $s3
        move $s0, $zero         # i = 0
for1tst: slt  $t0, $s0, $s3     # $t0 = 0 if $s0 ≥ $s3 (i ≥ n)
        beq  $t0, $zero, exit1  # go to exit1 if $s0 ≥ $s3 (i ≥ n)
        addi $s1, $s0, -1       # j = i - 1
for2tst: slti $t0, $s1, 0       # $t0 = 1 if $s1 < 0 (j < 0)
        bne  $t0, $zero, exit2  # go to exit2 if $s1 < 0 (j < 0)
        sll  $t1, $s1, 2        # $t1 = j * 4
        add  $t2, $s2, $t1      # $t2 = v + (j * 4)
        lw   $t3, 0($t2)        # $t3 = v[j]
        lw   $t4, 4($t2)        # $t4 = v[j + 1]
        slt  $t0, $t4, $t3      # $t0 = 0 if $t4 ≥ $t3
        beq  $t0, $zero, exit2  # go to exit2 if $t4 ≥ $t3
        move $a0, $s2           # 1st param of swap is v (old $a0)
        move $a1, $s1           # 2nd param of swap is j
        jal  swap               # call swap procedure
        addi $s1, $s1, -1       # j -= 1
        j    for2tst            # jump to test of inner loop
exit2:  addi $s0, $s0, 1        # i += 1
        j    for1tst            # jump to test of outer loop
```

Move params

Outer loop

Inner loop

Pass params & call

Inner loop

Outer loop

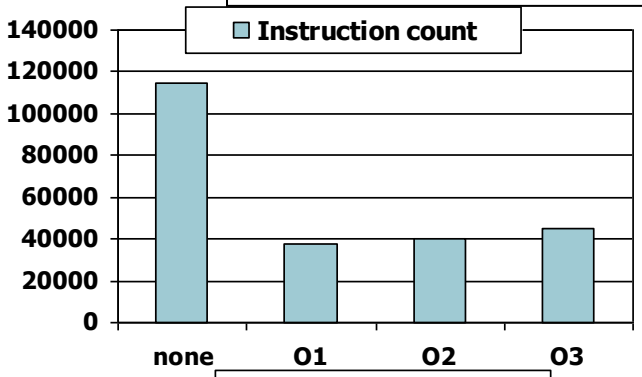


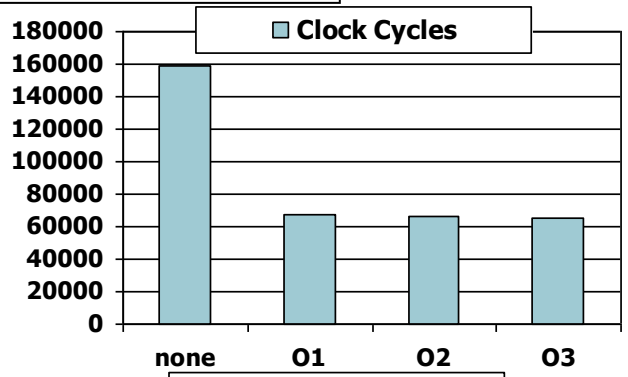
The Full Procedure

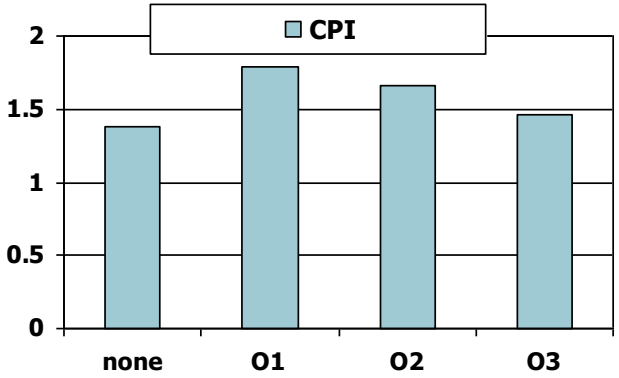
```
sort:    addi $sp,$sp, -20      # make room on stack for 5 registers
         sw $ra, 16($sp)        # save $ra on stack
         sw $s3,12($sp)         # save $s3 on stack
         sw $s2, 8($sp)         # save $s2 on stack
         sw $s1, 4($sp)         # save $s1 on stack
         sw $s0, 0($sp)         # save $s0 on stack
         …                      # procedure body
         …
         exit1: lw $s0, 0($sp)  # restore $s0 from stack
         lw $s1, 4($sp)         # restore $s1 from stack
         lw $s2, 8($sp)         # restore $s2 from stack
         lw $s3,12($sp)         # restore $s3 from stack
         lw $ra,16($sp)         # restore $ra from stack
         addi $sp,$sp, 20       # restore stack pointer
         jr $ra                 # return to calling routine
```

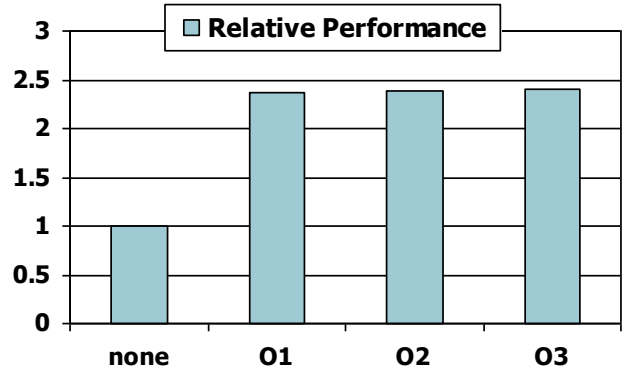
Effect of Compiler Optimization

Compiled with gcc for Pentium 4 under Linux



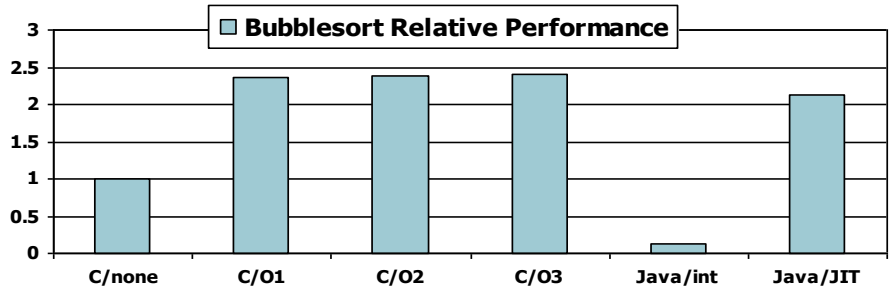


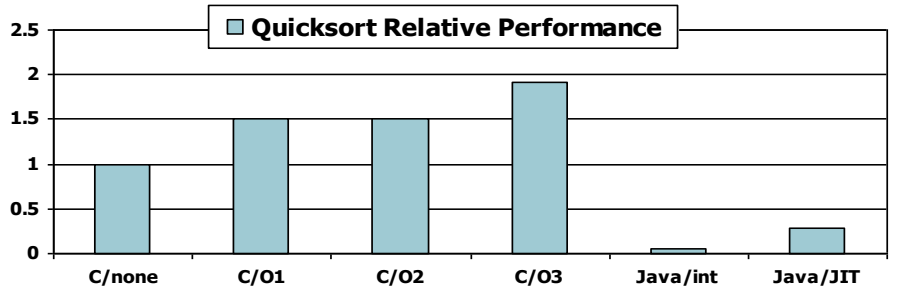


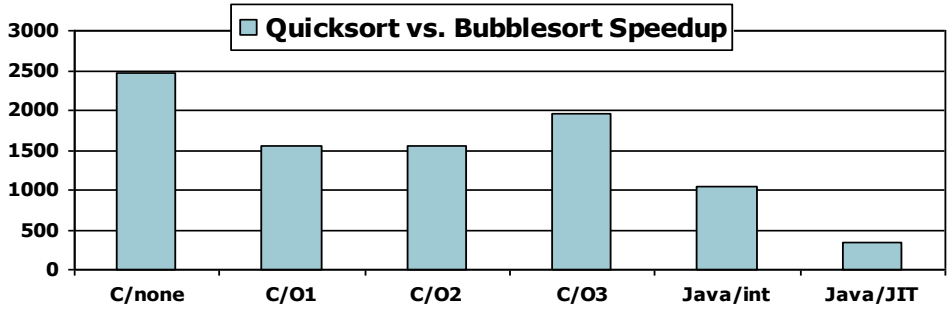


gcc optimization	Relative performance	Clock cycles (millions)	Instruction count (millions)	CPI
None	1.00	158,615	114,938	1.38
O1 (medium)	2.37	66,990	37,470	1.79
O2 (full)	2.38	66,521	39,993	1.66
O3 (procedure integration)	2.41	65,747	44,993	1.46

Effect of Language and Algorithm







$O(n^2)$ vs. $O(n\ logn)$

Language	Execution method	Optimization	Bubble Sort relative performance	Quicksort relative performance	Speedup Quicksort vs. Bubble Sort
C	Compiler	None	1.00	1.00	2468
	Compiler	01	2.37	1.50	1562
	Compiler	02	2.38	1.50	1555
	Compiler	03	2.41	1.91	1955
Java	Interpreter	–	0.12	0.05	1050
	JIT compiler	–	2.13	0.29	338

Lessons Learnt

- **Instruction count and CPI are not good performance indicators in isolation**
- **Compiler optimizations are sensitive to the algorithm**
- **Java/JIT compiled code is significantly faster than JVM interpreted**
  - Comparable to optimized C in some cases
- **Nothing can fix a dumb algorithm!**

Efficient algorithm is important!

Arrays vs. Pointers

CHAPTER2: INSTRUCTIONS: LANGUAGE OF THE COMPUTER

Arrays vs. Pointers

- **Array indexing involves**
  - Multiplying index by element size
  - Adding to array base address

Base address + Offset

- **Pointers correspond directly to memory addresses**
  - Can avoid indexing complexity

Memory Address

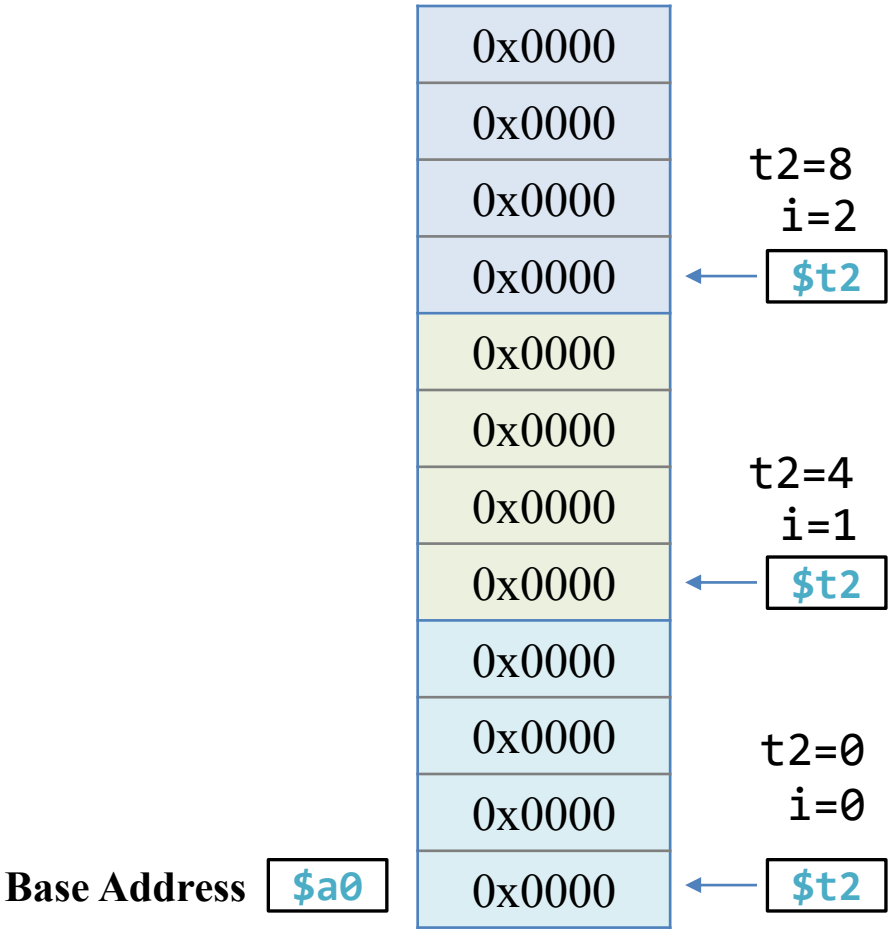
Example: Clearing1 - Array

```
clear1(int array[], int size) {
  int i;
  for (i = 0; i < size; i += 1)
    array[i] = 0;
}
```

```
       move $t0,$zero   # i = 0
loop1: sll $t1,$t0,2    # $t1 = i * 4
       add $t2,$a0,$t1  # $t2 = &array[i]
       sw $zero, 0($t2) # array[i] = 0
       addi $t0,$t0,1   # i = i + 1
       slt $t3,$t0,$a1  # $t3 =
                        #   (i < size)
       bne $t3,$zero,loop1 # if (…)
                           # goto loop1
```

Array[ ] $a0 **i** $t0

Size $a1



Example: Clearing2 - Pointer

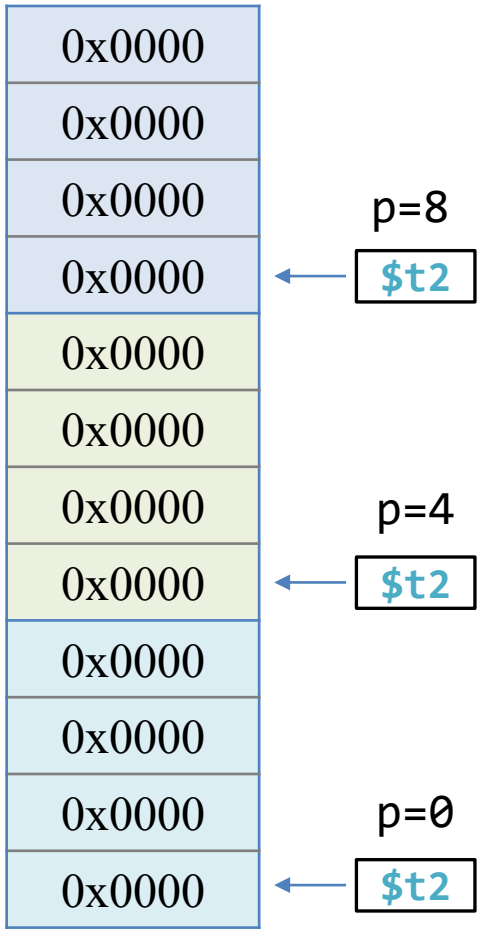
```
clear2(int *array, int size) {
  int *p;
  for (p = &array[0]; p < &array[size];
       p = p + 1)
    *p = 0;
}
```

```
       move $t0,$a0    # p = & array[0]
       sll $t1,$a1,2   # $t1 = size * 4
       add $t2,$a0,$t1 # $t2 = &array[size]
loop2: sw $zero,0($t0) # Memory[p] = 0
       addi $t0,$t0,4  # p = p + 4
       slt $t3,$t0,$t2 # $t3 =
                       #(p<&array[size])
       bne $t3,$zero,loop2 # if (…)
                           # goto loop2
```

Array[ ] $a0 **p** $t0

Size $a1

Calculate boundary



Example: Clearing and Array

```
clear1(int array[], int size) {
  int i;
  for (i = 0; i < size; i += 1)
    array[i] = 0;
}
```

```
clear2(int *array, int size) {
  int *p;
  for (p = &array[0]; p < &array[size];
       p = p + 1)
    *p = 0;
}
```

```
       move $t0,$zero   # i = 0
loop1: sll $t1,$t0,2    # $t1 = i * 4
       add $t2,$a0,$t1  # $t2 =
                        #   &array[i]
       sw $zero, 0($t2) # array[i] = 0
       addi $t0,$t0,1   # i = i + 1
       slt $t3,$t0,$a1  # $t3 =
                        #   (i < size)
       bne $t3,$zero,loop1 # if (…)
                           # goto loop1
```

```
       move $t0,$a0     # p = & array[0]
       sll $t1,$a1,2    # $t1 = size * 4
       add $t2,$a0,$t1  # $t2 =
                        #   &array[size]
loop2: sw $zero,0($t0)  # Memory[p] = 0
       addi $t0,$t0,4   # p = p + 4
       slt $t3,$t0,$t2  # $t3 =
                        #(p<&array[size])
       bne $t3,$zero,loop2 # if (…)
                           # goto loop2
```

Comparison of Array vs. Ptr

- **Multiply "strength reduced" to shift**
- **Array version requires shift to be inside loop**
  - Part of index calculation for incremented i
  - c.f. incrementing pointer
- **Compiler can achieve same effect as manual use of pointers**
  - **Induction variable elimination**
  - Better to make program clearer and **safer**

Take multiply to shift

Reduced array address calculations

ARM and x86 instruction sets

CHAPTER2: INSTRUCTIONS: LANGUAGE OF THE COMPUTER

ARM & MIPS Similarities

- **ARM: the most popular embedded core**
- **Similar basic set of instructions to MIPS**

	ARM	MIPS
Date announced	1985	1985
Instruction size	32 bits	32 bits
Address space	32-bit flat	32-bit flat
Data alignment	Aligned	Aligned
Data addressing modes	9	3
Registers	15 × 32-bit	31 × 32-bit
Input/output	Memory mapped	Memory mapped

The Intel x86 ISA

- **Evolution with backward compatibility**
  - 8080 (1974): 8-bit microprocessor
    - Accumulator, plus 3 index-register pairs
  - 8086 (1978): 16-bit extension to 8080
    - Complex instruction set (CISC)
  - 8087 (1980): floating-point coprocessor
    - Adds FP instructions and register stack
  - 80286 (1982): 24-bit addresses, MMU
    - Segmented memory mapping and protection
  - 80386 (1985): 32-bit extension (now IA-32)
    - Additional addressing modes and operations
    - Paged memory mapping as well as segments

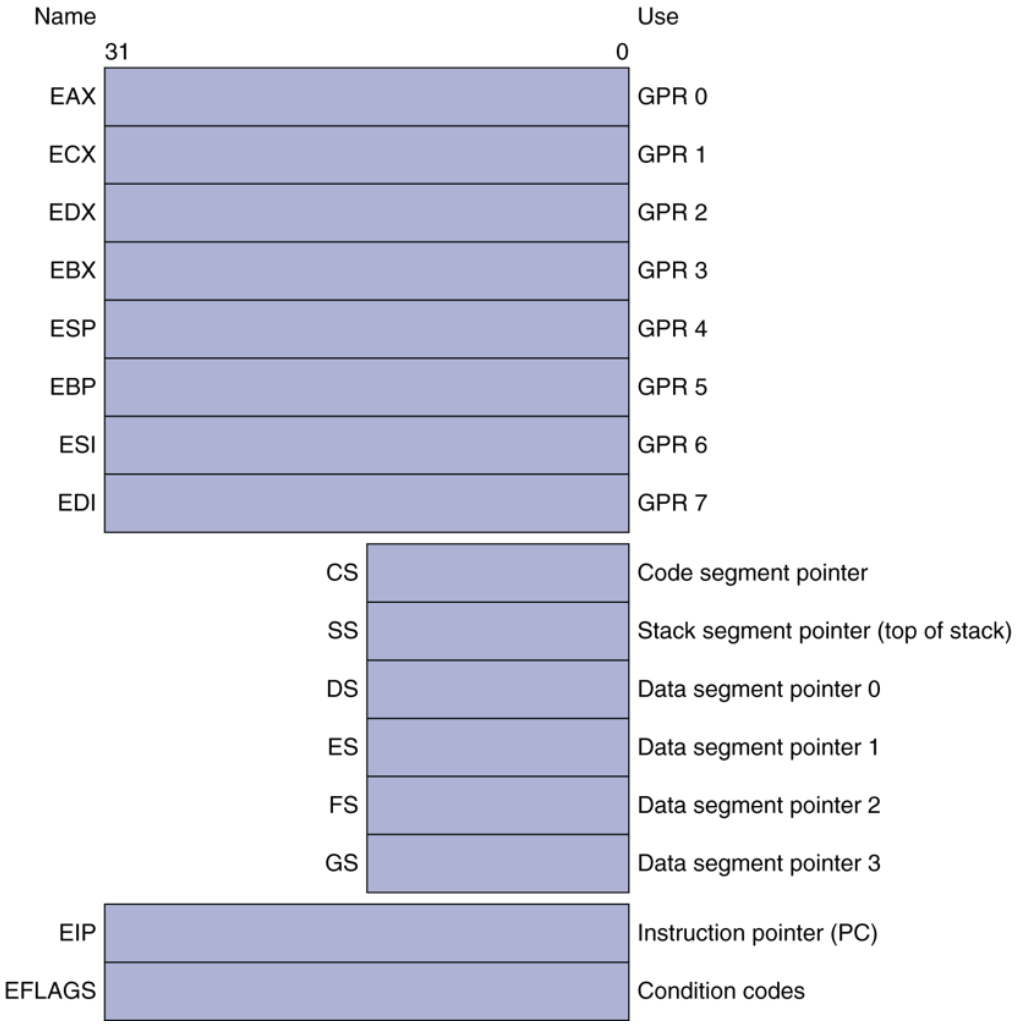
The Intel x86 ISA

- **Further evolution…**
  - i486 (1989): pipelined, on-chip caches and FPU
    - Compatible competitors: AMD, Cyrix, …
  - Pentium (1993): superscalar, 64-bit datapath
    - Later versions added MMX (Multi-Media eXtension) instructions
    - The infamous FDIV bug
  - Pentium Pro (1995), Pentium II (1997)
    - New microarchitecture (see Colwell, The Pentium Chronicles)
  - Pentium III (1999)
    - Added SSE (Streaming SIMD Extensions) and associated registers
  - Pentium 4 (2001)
    - New microarchitecture
    - Added SSE2 instructions

The Intel x86 ISA

- **And further…**
  - AMD64 (2003): extended architecture to 64 bits
  - EM64T – Extended Memory 64 Technology (2004)
    - AMD64 adopted by Intel (with refinements)
    - Added SSE3 instructions
  - Intel Core (2006)
    - Added SSE4 instructions, virtual machine support
  - AMD64 (announced 2007): SSE5 instructions
    - Intel declined to follow, instead…
  - Advanced Vector Extension (announced 2008)
    - Longer SSE registers, more instructions
- **If Intel didn't extend with compatibility, its competitors would!**
  - Technical elegance ≠ market success

Basic x86 Registers



Basic x86 Addressing Modes

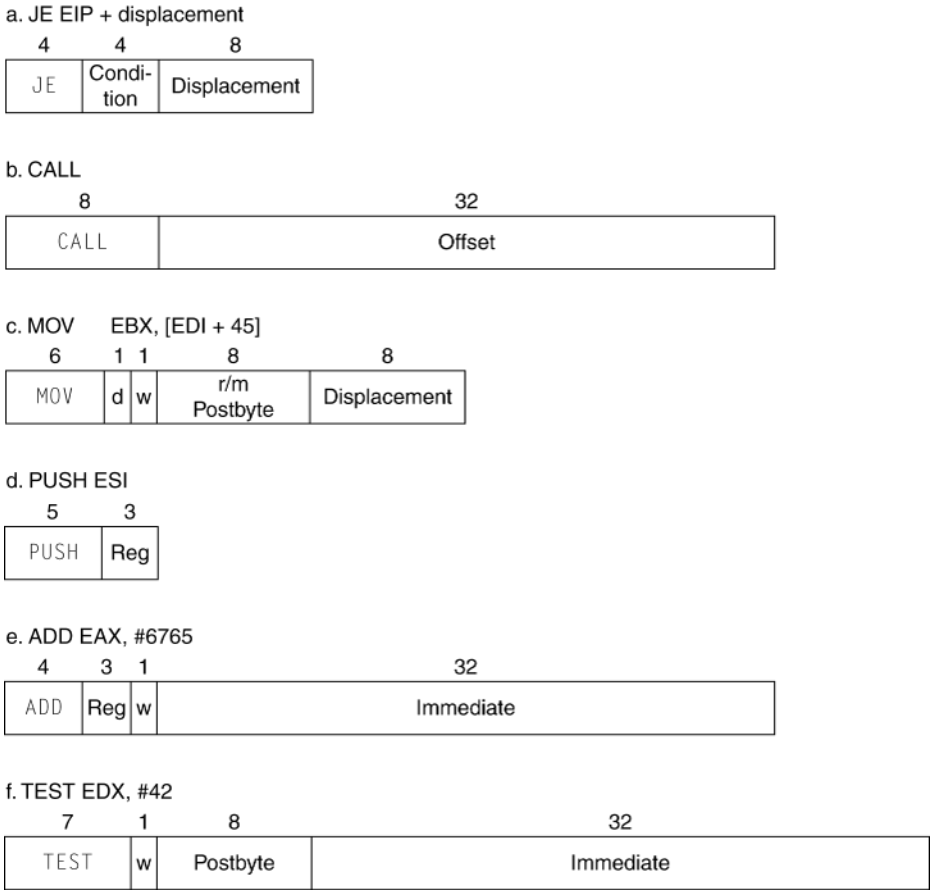
- **Two operands per instruction**

Source/dest operand	Second source operand
Register	Register
Register	Immediate
Register	Memory
Memory	Register
Memory	Immediate

- **Memory addressing modes**
  - Address in register
  - Address = Rbase + displacement
  - Address = Rbase + 2scale × Rindex (scale = 0, 1, 2, or 3)
  - Address = Rbase + 2scale × Rindex + displacement

x86 Instruction Encoding

- **Variable length encoding**
  - Postfix bytes specify addressing mode
  - Prefix bytes modify operation
    - Operand length, repetition, locking, …



Implementing IA-32

- **Complex instruction set makes implementation difficult**
  - Hardware translates instructions to simpler microoperations
    - Simple instructions: 1–1
    - Complex instructions: 1–many
  - Microengine similar to RISC
  - Market share makes this economically viable
- **Comparable performance to RISC**
  - Compilers avoid complex instructions

Fallacies and Pitfalls

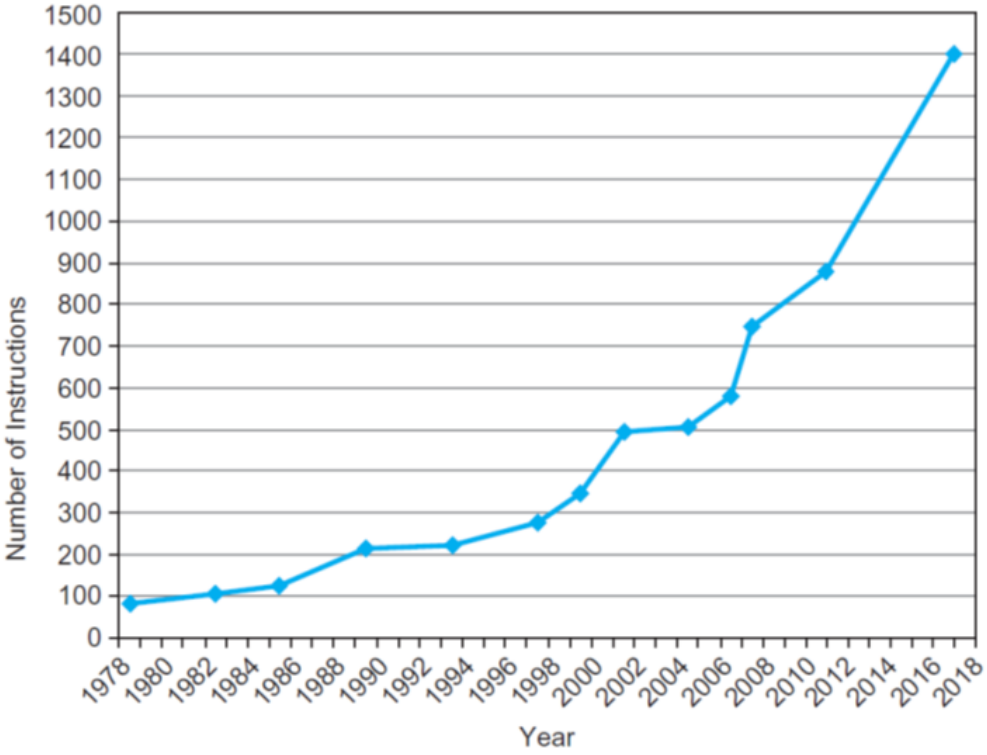
CHAPTER2: INSTRUCTIONS: LANGUAGE OF THE COMPUTER

Fallacies (謬誤)

- **Powerful instruction ⇒ higher performance**
  - Fewer instructions required
  - But complex instructions are hard to implement
    - May slow down all instructions, including simple ones
  - Compilers are good at making fast code from simple instructions
- **Use assembly code for high performance**
  - But modern compilers are better at dealing with modern processors
  - More lines of code ⇒ more errors and less productivity

Fallacies (謬誤)

- **Backward compatibility ⇒ instruction set doesn't change**
  - But they do increase more instructions



x86 instruction set

Pitfalls

- **Pitfall: Sequential words are not at sequential addresses**
  - Increment by 4, not by 1!
- **Pitfall: Returning a Pointer to a Local Variable**
  - Local variables may be stored in the stack frame.
  - The stack frame is released when the function returns.
  - The pointer becomes dangling. (pointer invalid)

Concluding Remarks

CHAPTER2: INSTRUCTIONS: LANGUAGE OF THE COMPUTER

Concluding Remarks

- **1. Instructions are the language of hardware**
  - MIPS uses simple, regular instructions.
  - Registers hold operands; memory stores data.
- **2. Programs are built from instructions**
  - Arithmetic, memory access, and control flow.
  - Procedures use registers and the stack to manage resources.
- **3. ISA design involves trade-offs**
  - Simplicity favors regularity.
  - Smaller is faster.
  - Make the common case fast.
  - Good design demands good compromises.
- **4. Software and hardware work together**
  - High-level code → Assembly → Machine code → Hardware.